

The OADA System

ZYGOMEB
Optim Finance
zygomeb@gmail.com

Document Version: 2.0

System Version: 2.0

2025-11-20

For the latest version visit [THIS REPOSITORY] .

The document captures the current state of the deployment of the system, sufficient reasoning and explanation of the system operations, as well as the changelog from past versions.

It is presented as-is for purely informational purposes.

A system labeled as an Algorithmic Market Operation (AMO) is a Strategy equipped with the power to manipulate the supply of OADA by minting and burning using associated rules.

Changelog V1 -> V2 (Forward, V2)

This section is an expansion unto the functionality of V1 described in the Core+Base section, which should be read first. Only afterwards, does this section make sense, as it builds on top and overwrites the functionality from prior.

Assuming a successful ODAO vote, the system is updated in-place, and without the need to migrate liquidity. All prior integrations and tokens are the same as before.

Sections 1-5 are unchanged, and in section 6 there were the new Forward package modules described in brief. Otherwise, only this section is necessary to be read assuming familiarity with the V1 system. V2 is a 'growth on top' of V1, in a sense that none of the prior schedule is impacted, and the system now gets a sequencer managed loan underwriting ability that otherwise leaves all of the prior behavior untouched.

The changes brought alive with this update fulfill the 'minimum self justifiable' version of OADA that has merit enough alone to be a self-sustainable and growing to scale system. Initially intended to launch much sooner, however, with unforeseen obstacles in our way of our initial plans, we needed to wait for the release of FluidTokens V3, as a pivot from our original, Liqwid-based roadmap. It has come with added benefits of full control of the collaterals without deference to governance of pooled permissions set by the Liqwid DAO. With the feature centered on FluidTokens V3 we have arrived at a strictly superior product with the ability to lend exactly on our terms, with much better stability and redemption guarantees.

Lending AMO via FluidV3

We add one new module to the system, that issues OADA against ODAO whitelisted collateral (Asset+LTV+MaxCredit+Oracle+MaxDuration) with sequencer controlled interest rates on the FlluidTokensV3 platform. The system permits only the creation of fixed duration loans, with the ability to refinance loans being provided, in order to extend the credit indefinitely. The system has a whitelisted maximum amount of credit that can be issued against one given asset, which means that in any given period (one month being the default duration) it permits only part of the maximum to be lent out (or available to be borrowed) in order to make liquidity available for refinancing.

These parameters are controlled (changing, adding new markets) by a full ODAO vote and executed by the ODAO Core multisig to update the system afterwards.

Importantly, with this addition, there is a real, non-zero chance that the system operations now undergo a partial loss, or even a total loss, depending on the amount lent out. The sOADA holders are now fully in their place as Junior Tranche of the system, taking on all profit bar fees and all loss. We are introducing a 7 day unstaking delay to the system in order to prevent any bank run scenarios, and allow us to take the minimum necessary precautions and risk management as capital cycles in and out of the system.

Credit Operations

The system dynamically adjusts total amount available to be borrowed by supplying new liquidity to specifically enable refinancing by each individual loan 2 days before the loan expires, expanding supply further to meet demand in that timeframe. If other borrowers pick it up instead, the system adds further capacity automatically.

Individual substantial borrowers are encouraged to work with the ODAO to communicate intent, and we make reserve of further capacity in the total maximum lent out in order to facilitate enabling of transaction creation in order to make any denial of refinancing for significant borrowers impossible.

At the end of the duration, the system pushes available amount of liquidity that was borrowed to enable refinancing – without the ability to give to the specific borrower, or even an existing one the ability to refinance. These ‘refinancing’ loans are available for everyone to pick, and if not picked up by the borrower, force repayment by end of duration, or liquidation.

Liquidations

Liquidations in the system gradually sell collateral into Splash pools directly (both ADA/X and OADA/X). The following is an example of the first collateral asset, SNEK, and its liquidation description. It should be thought of as the default for high valuation assets. Planned in the short term future are other assets of similar risk management strategy. While the system can in the future be configured to have only time based liquidation (no repayment was made by end of term), currently we have both price based, and time based liquidations, which proceed both in an identical fashion.

The pricing starts at oracle price and decays exponentially by a fixed % each time interval. The default liquidation model is 24 hour liquidation total, decaying by 3% each step (10 minutes), with the expectation being that the liquidation is processed in the first 6 steps, and LTVs are placed such that at after that point the system starts to sell at a loss (150% LTV, assuming a further 20% drop during that timeframe). The model is chosen to be able to handle large liquidations with ample time for arbitrage and dip buying. Of course, presented is the default model to be used for SNEK as a first collateral pilot asset, and it may change for other assets.

Impact on Peg Stability

Obviously, for example if the system contains 6M ADA total, 5M of which is in the sOADA tranche, then the maximum amount of losses than can be covered before the system loses backing of OADA is 5M, zeroing out sOADA. This should be thought of as a guiding stick for our operations.

Obviously, also, if there are, in this same example, 3M OADA loans, which then sell on the open market, then the system cannot buy back all of the OADA if it were to be all unstaked until the loans are repaid. (That is, 3M OADA get sold for 3M ADA, leaving 3M ADA in the system. if 4M sOADA then wanted to withdraw, the system would not have the capacity to maintain the peg). This is normal behavior, and we expect (as in the case of staking strategy) there to be moments where temporarily the open market price is impacted.

At those times the system will be forced to decrease the amount of its liabilities by not providing a refinancing line to the outstanding loans. This is a hard incentive for the borrowers to either be liquidated, or close their positions, and unlike indefinite length loans, allows us to always eventually (with bounded time of max loan duration, i.e. currently 1 Month) recover the open market price of the OADA token. If suspected to be done out of market manipulation to force high value individual borrowers to close or be liquidated, we intend to provide refinancing support to prevent such situations from occurring, either by loans from reserve capacity, capacity expansion if possible, or lending out of ODAO treasury.

To explicitly reiterate, as per design, under stress, the system may be unable to restore the open market value of OADA for upwards of a month in theory as designed, forcefully contracting the liabilities without allowing refinancing. In practice, it is expected to be much shorter, as the discount on OADA is highly prized for arbitrage, and repayment of loans are thus much favorable. Prior to V2, this period was just one epoch, due to the Stake Auction operations.

Thus, as shown, we arrive at proof of the claim that the value of the OADA asset is always possible in fixed amount of time to be restored to its intended price range. The term structure allows us to, with certainty, collapse the liabilities outstanding, without compromising borrower's ability to plan ahead of time (no immediate redemption).

Interest Rates

As a measure to help bootstrap the credit facility the interest rates are chosen as static 7% borrow rate. For the next few months, there will be no change to that. With that overall guidance, we pass the ability to change this policy to ODAO Council.

The rates accrue to sOADA as profit when repayment/refinancing happens, as well as liquidation on epoch boundaries. Any negative profits are merged instantly to avoid bank runs.

Conclusions

Aiming to strike the balance between reasonable management of collateral pricing and efficiency, with intention to be the best option available on the market for borrowing against large amounts of tokens, we have crafted a system flexible enough to be able to underwrite and adapt to just about all customers.

We may, if the borrowers want to utilize, or come to utilize, OADA, as not an intermediary asset to simply sell into ADA, and instead provide liquidity on a DEX as an example, provide a designated line of credit.

The time will steer the direction of further configuration and development based on market demand and ODAO input.

1. High Level Description (Core + Base, V1)

Shadow looms over the ghost of an ethereal beauty, a raw force of financial warfare; form of a spinless cutthroat efficiency, taking root between the cracks of those that take their power for granted. Harnessed, for now. Made tame, able to be understood. Made humane.

OADA System is an instantiation of the OTOKEN framework to create a fully-backed ADA derivative token. It is currently range-pegged within one percent of 1:1, and rebalances the open market price of OADA to keep the price of the asset within that range via buybacks and arbitrage.

Autocompounding and able to take advantage of ecosystem yield opportunities, the system passes their fruits on to the staked version – that is, both taking on the risk of the strategies as well as the

yield from them. It is intended and structured to be as low risk as possible, limited in intent to only smart contract and similar low level lending risks.

Allocation of system reserves and invocation of the majority of AMOs requires a signature of one of the system operators referred to as a Controller. It is a governance-delegated position, that possesses no custody of the assets, in all situations aside from a potential system exploitation.

The OADA system is governed by the ODAO, via the use of a Soul Token that can upgrade it in any way necessary.

2. Status Report

This section we will contain high level milestones, as well as highlight points that show the system's history.

The system version described in this document is 2.

3. Relevant Information

All of the information in this section contains important context, extrinsic to the system

The ODAO treasury has deposited TODO ADA and is keeping it unstaked to boost the initial adoption of the system, as well as providing TODO paired with equal amount of ADA to keep a long term liquidity base for the system's operations.

OADA Soul Token is currently held in a multisignature wallet between Optim Labs members and ODAO Council members. It is intended to be transferred over to a fully decentralized governance system once one is built up, fit for the protocol's requirements.

4. System Solvency Analysis and Risks

When it comes to the risk profile of the system, for actions that mint OADA (rules) we aim to ensure the highest degree of safety of the system. Here we compile the risks for counterparties of the OADA system and their risks, for every strategy-type AMO, as well as arguments for what makes them safe to execute.

Further elaboration on each strategy is given in their own sections, here only outlining the key risks of them.

4.1. DEX AMO Risks

- Currently only interfacing with the Splash OADA/ADA stableswap pool
- The Splash system is quite new, and without much capital in it and therefore not battle-tested
- Splash DAO governance can tweak fee parameters on the pool and potentially cause the system principal loss
- The Stableswap pool is open source and audited

Strategy Added Risks

4.1.1. Peg Protection Swaps

The system guards against making a transaction that is unprofitable. Meaning, it is impossible for the system to lose money through swapping between OADA and ADA.

4.1.2. Liquidity Provision

It can be accurately described, from another perspective, as a leveraged (~2x) LP strategy with no cost of leverage.

We use the commonly known fact that the sum of tokens deposited does not decrease (and in fact, increases) away from the center point of the curve for any amplification parameter of the canonical stableswap.

This implies that, for deposits that are made at a price point closer to a 1, if the price goes further away from the 1, the sum of assets backing the LP increases. An important feature for OADA as we can take advantage of the symmetry of the stableswap curve in combination with system accounting for any OADA profits as = 1 ADA (and conversely), to assert that by depositing liquidity in the 1:1 center of the AMM, can never lose money via rebalancing loss (also can be called, the no impermanent loss property).

We want to, however, to be able to deposit *not at the perfect center*, so that only a minimum amount of reserves are used to rebalance the pool prior. The mathematical proof doesn't hold for such deposits, and we're forced to refer to our numerical analysis that shows that with a swap fee of 0.1% is sufficient to guarantee that no rebalance can cause the final withdrawn sum to go under the initial sum of deposited assets.

Furthermore, in presence of a swap fee, it may be impossible to rebalance the pool to the perfect center without taking a loss on the rebalancing swap. This analysis was done for 100, 150 and 200 amplification argument stableswap pools.

A constraint on deposits is that the pool doesn't sell 1 OADA for less than 1 ADA, so we are limited to deposits in range of prices above 1-swapfee. The final safe deposit range for a 0.1% fee pool going to the LP is the price range of 0.999-1.001 ADA per OADA.

The above range is the mathematically perfect range. Some error between a nonzero splash protocol fee and a fractional number error safety buffer gives us a slightly tighter range. Which implies, also, that with the current restrictions on arbitrage swaps (no loss on swap), there exists a very small price range where the protocol is neither able to deposit, nor bring it to a price point at which it will be able to.

Such states are not realistically profitable for any potential attacker to exploit, nor is it realistically possible to keep an open market price perfectly within the very tight (0.05%) TODO inactive range. Even if such an attacker is assumed to be Byzantine, the system is under no risk if this part of the DEX AMO strategy is not able to execute.

It goes without saying that the withdrawal counterpart of the equation does not have any such limits.

4.2. Staking Auction AMO Risks

- This is a strategy providing liquidity to a market made as a dapp internal to the OADA system considered part of the AMO as a whole

Strategy Added Risks

- Gradually locks up system ADA reserves during the epoch, and provided users are willing to pay a high price, they can acquire all of the reserves of the system.
 - The pricing curve gets increasingly steep, upwards of paying double and even quadruple digit APY for a single epoch in order to acquire all of the reserves early in the epoch
 - It is still, however, possible to acquire. The system is very much prepared to accept any such offers.
- Funds locked up aren't able to be used in other strategies

- Noteworthy, also not able to be used to protect the peg of the OADA/ADA pair. As such, it can be expected that at such times, without users willing to acquire it under peg, open market price will be totally steered by speculation.
- Upon the start of the new epoch, the assets are released, and able to be used to perform any function, including a repegging of the pair
- Oracles during such market conditions can and will be affected and systems not prepared to deal with such system operations should not be used to acquire leverage on assets trading against OADA or against OADA or sOADA themselves.

4.3. Staking AMO Risks

- Unlike the other modules in this section, this is a note about risks associated with using the Staking AMO (staking/unstaking OADA for/to sOADA)
- The main risk associated with staking OADA is that in the case of a system loss – it bears the cost of the loss. It can be thought of as the Junior tranche, that gets all the yield bearing the risk of covering for system loss.
- System loss under normal operations should not occur as the system takes on a minimized risk exposure.

5. Capital Flows

In this section we break down the system's operations as performed by the currently delegated Optim Labs Controller.

5.1. Schedule and Operations

For OADA to run smoothly and transparently we must have well-defined semantics for each of its actions, as well as perform them. As a simple mental model of how the capital flows through the system, the ADA gets deployed into strategies during the epoch and at the end of it, the staking rights are sold to the highest bidder. After the epoch boundary, all of that ADA gets redeployed back into the strategies.

5.1.1. Deposit-Deploy

As users mint OADA with ADA, that ADA sits at the Deposit Address waiting to be processed. Processing happens either at the end of the epoch, or once enough of it minted. This is done in order to have the controller save on transaction fees without losing any meaningful amount of profit for the system.

The current threshold for the Controller to process deposits is 100 000 ADA in total pending.

When a Deploy routine is triggered, whether by a deposit threshold or by beginning-of-epoch redeployment, the system looks up the current distribution of assets and distributes the inflow of capital targeting up to a 2% deviation from the 'ideal' set distribution between strategies. This is purely a Controller guideline, and it has the freedom to act outside of it and distribute it as it believes is most beneficial.

Currently the distribution target is 100% to the DEX AMO, as there are no other strategies deployed.

5.1.1.1. DEX AMO Deploy

DEX AMO operates by keeping, for the sake of efficiency, a small part of the total funds (10%) it controls idle, and waiting to be used for peg keeping by OADA buybacks. This number will be revised in the future to ensure high degree of efficiency.

If the AMO controls 0 ADA right now (first deploy of the epoch) then a purchase of OADA is made to bring it up to 0.999 ADA per OADA price. This amount of capital is pre-calculated during

distribution between strategies and isn't included in the 30% the DEX AMO gets, but rather the 30% is calculated after this amount is subtracted from the total reserves.

What the purchase allows, is a safe deployment of ADA paired with minted OADA in the exact same ratio as the DEX pool. Assuming that the swap fees are 0.1% per swap or above, this makes depositing of minted OADA in this ratio to ADA deposited an action that cannot lower the backing of it from 1:1, as any ADA entering a swap will not cause a leak of OADA acquired for less than 1 ADA unless someone sells it for less than that first, which balances itself out entirely.

5.1.2. Clearing-Rate-Match Event

The Stake Auction of OADA is done continually all throughout the epoch until it is time to Undeploy-Auction the rest of the system reserves – this way, with a Continuous Dutch Auction, we are able to minimize the ability for bidders to strategically place orders and ensure the highest profitability for the system given demand for stake.

To determine the current price of stake, the controller works with a pricing curve based on the time passed in epoch, and remaining reserves to be sold – full equation in its section. It uses a minimum match size of 250k ADA stake and matches bids on a FCFS basis sorted first by the APY offered.

When enough bids to get a minimum size match are found, such that the system, after matching them, will reprice clearing rate to be at least as much as it has just received – the system will match these orders and lock the stake with their chosen stake key until the epoch ends.

5.1.3. Undeploy-Auction

Before the epoch ends, an undeploy event takes place, which pulls all available assets out of the system. This is also the time when the system guarantees profits to be synced and pushed to affect the sOADA/ADA exchange ratio.

The ADA is then moved to fill stake bids (which were placed and filled all throughout the epoch) in the Stake Auction AMO, where it resides up until the end of the epoch, and afterwards syncs profits from the auction and moves to deploy the funds in the rest of the system. In other words, settlement of the actual auction takes place within one hour before the epoch ends.

5.1.4. Pegkeeper-Capitalize Event

Unlike the purchase of OADA during the deployment at the start of the epoch, during the epoch the system only protects the peg when it deviates under and up to 1% down from the 1:1 status.

5.1.5. Voltaire Notes

The system allows Voltaire vote power to be acquired, as it is derived from one's stake key weight. Given how the governance process of Voltaire works, everyone will know 1 epoch ahead of time if the next epoch has any important votes going on. Equivalently, there can be no last minute surprises to ratify a proposal without it going through at least 2 epoch boundaries first (1 boundary to allow voting, and the next boundary to see if it ratified or otherwise).

5.1.6. Catalyst Snapshot Event

If a Catalyst vote power snapshot is to take place, and it is taken close to an epoch boundary, the system has to decide whether to auction off the stake power OR keep it for the ODAO. An example (as currently it is not yet decided) policy is to only sell to bids offering 10% APY or higher for that epoch boundary. If it is not close to a boundary, then the system will simply capture all of the vote power for the ODAO.

When vote power is captured by the ODAO, the ADA rewards for voting will be passed into the OADA system using the Donation Strategy, including the stake rewards, if on an epoch boundary.

Note: Given the centralized nature of the snapshot (especially when no exact snapshot slot is given) the Controller cannot guarantee that this operation will be carried out as described here.

6. Active Modules

The active modules, for the sake of categorization and convenience are grouped into Packages. Currently there are two packages, the Core and the Base packages.

Within Core is the minimal version of the system containing all of the accounting operations logic. It is audited by Anastasia Labs and fully open sourced

Base Package contains the necessary AMOs for the system's operations including interoperation with closed source systems like Liqwid. It is neither audited nor open sourced, as doing so requires all of the systems it integrates with to open source first for inspection of the auditors. In case of any problems found in a potential future audit, once possible, the system can gracefully upgrade them via governance.

6.1. OADA Token

Introduced in	1.0
Last Revision	1.0
Package	Core
Source	https://github.com/OptimFinance/clean-code
Audits	https://github.com/OptimFinance/clean-code
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
OADA Token is an extensible set of rules for minting and burning OADA. It is governed by a soul token and intrinsically has no ties with the rest of the system beyond what rules define. One could, in theory define additional rules that interact with components completely external to 'the system' as we speak here – it is easily observed that this method of defining a token makes it possible to define a completely asynchronous and everpresent asset with rules that span the entire market.	

6.1.1. OADA Rule Whitelist

Introduced in	1.0
Last Revision	1.0
Package	Core
Source	https://github.com/OptimFinance/clean-code

Audits	https://github.com/OptimFinance/clean-code
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
First mention of this pattern in this document, additional elaboration of the pattern is given. The whitelist is a set of fractioned data that is used to authenticate and prove witness to a governance process having been processed successfully, that is, a witness of a soul token was present upon the creation of a whitelist record as a mint of a whitelist NFT is made only possible by that. Here we use this pattern to add 'Rules', aforementioned fractional invocations that define when it is possible to mint and burn OADA. It is done via the OADA policy looking for an authenticated self NFT-holding reference utxo that has a datum with a script that is being invoked as a withdrawal. And if that is present, anything is permitted.	

6.1.2. OADA Whitelisted Rules

In this section we record the currently deployed rules for minting/burning OADA. They are explained under the AMO that is related to them, to give necessary context to their functionality.

6.1.2.1. OADA Deposit Rule

6.1.2.2. OADA/sOADA Staking Rule

6.1.2.3. OADA Fee Claim Rule

6.1.2.4. OADA Splash DEX Rule

6.1.2.5. OADA Stake Auction Rule

6.2. sOADA Token

Introduced in	1.0
Last Revision	1.0
Package	Core
Source	https://github.com/OptimFinance/clean-code
Audits	https://github.com/OptimFinance/clean-code
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH

In its technical design, sOADA is equivalent to OADA. It is only the presence of different rules that gives the semantical meaning to the token.

6.2.1. sOADA Rule Whitelist

Introduced in	1.0
Last Revision	1.0
Package	Core
Source	https://github.com/OptimFinance/clean-code
Audits	https://github.com/OptimFinance/clean-code
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
As above.	

6.2.2. OADA Whitelisted Rules

In this section we record the currently deployed rules for minting/burning sOADA. They are explained under the AMO that is related to them, to give necessary context to their functionality.

6.2.2.1. OADA/sOADA Staking Rule

6.3. Deposit AMO

Introduced in	1.0
Last Revision	1.0
Package	Core
Source	https://github.com/OptimFinance/clean-code
Audits	https://github.com/OptimFinance/clean-code
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
The Deposit AMO is the simplest possible system, that also illustrates the bottom line of the economic system model. It allows for 1 ADA that enters the AMO (and into the system), 1	

OADA to be minted. As a general rule of thumb, this is the simplest and most robust way to ensure a peg can be kept.

It can, however, be expanded to the notion of overcollateralized, allowing it to be issued against collateral instead. This, as seen by other protocols such as Maker, is quite a profitable business. It must, however, be done with caution as overissuance of tokens in this way without ensuring the stability of the asset will hold price on the open market will lead to price target (peg) failure.

Currently the system has no overcollateralized avenues to issue the token.

6.3.1. OADA Deposit Rule

Introduced in	1.0
Last Revision	1.0
Package	Core
Source	https://github.com/OptimFinance/clean-code
Audits	https://github.com/OptimFinance/clean-code
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
Deposit Rule simply allows minting of X OADA if X ADA is sent to the Deposit AMO. It has a minimum of 100 OADA minted to prevent dusting attacks.	

6.4. Staking AMO

Introduced in	1.0
Last Revision	1.0
Package	Core
Source	https://github.com/OptimFinance/clean-code
Audits	https://github.com/OptimFinance/clean-code
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH

All of the accounting within the contracts is headed down a pipeline from the strategies through the CM AMO to the Staking AMO. It tracks and accounts for the DAO Fee, and distribution of it to the sOADA holders by way of changing the sOADA/OADA exchange rate.

Of importance are the **odao_fee** and **staking_cap** parameters that can be changed by governance, and dictate a % cut of total system profit routed towards the DAO treasury and the maximum amount of sOTOKENS.

The ODAO fees accumulate in sOADA and circumvent the cap on staked amount upon accrual, and may be claimed anytime by the fee claimer token, the only governance witness token of the system other than the soul token itself.

6.4.1. Batch Stake Request

Introduced in	1.0
Last Revision	1.0
Package	Core
Source	https://github.com/OptimFinance/clean-code
Audits	https://github.com/OptimFinance/clean-code
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
Using a direct method of accessing the Staking AMO utxo was originally planned, but for reasons of accessibility and spam prevention – as well as enabling the creation of a stake queue, we introduce a necessary intermediation script that is to be processed by the Controller in a FIFO manner.	

6.4.2. OADA/sOADA Staking Rule

Introduced in	1.0
Last Revision	1.0
Package	Core
Source	https://github.com/OptimFinance/clean-code
Audits	https://github.com/OptimFinance/clean-code
Plutus Version	2
Unparameterized Script Hash	HASH

Deployed Script Hash	HASH
The rule is used both by sOADA and OADA, and guards the staking and unstaking process. Specifically, it ensures that mints of sOADA happen only when OADA is burned in a transaction that has the Staking AMO, and that the mint is done at the current sOADA/OADA exchange rate, as well as that it was indeed recorded in the total staked amount. The opposite way, to unstake/burn sOADA and mint OADA, the same process takes place, with an unstake fee of 0.1% of the total.	

6.5. Collateral Management AMO

Introduced in	1.0
Last Revision	1.0
Package	Core
Source	https://github.com/OptimFinance/clean-code
Audits	https://github.com/OptimFinance/clean-code
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
The Collateral Management AMO is at the heart of the system, from it all the AMOs flow and report to, accounting of profits and potential losses; the CM AMO is recording all the created instances of whitelisted strategies to allow them free capital flow and accounting through all of the system's tendrils.	

6.5.1. NFT ID Policy

Introduced in	1.0
Last Revision	1.0
Package	Core
Source	https://github.com/OptimFinance/clean-code
Audits	https://github.com/OptimFinance/clean-code
Plutus Version	2
Unparameterized Script Hash	HASH

Deployed Script Hash	HASH
A utility token that allows the creation of a provable globally unique non fungible asset. Used to identify strategies by their IDs, and most importantly, the CM AMO by its ID that in turn defines the entire system.	

6.5.2. Controller Whitelist

Introduced in	1.0
Last Revision	1.0
Package	Core
Source	https://github.com/OptimFinance/clean-code
Audits	https://github.com/OptimFinance/clean-code
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
A whitelist, governed by ODAO, that holds the PubKeyHashes of Controllers. For many of the system transactions, a signature by one such actor is required. Context as to why they must be guarded is simple: a decision problem and guarantee of a schedule (as well a definition of such schedule) would need to otherwise be put onchain to guard against malicious actions that can't lose the system principal but can otherwise make the system behave erratically and cause disruption in the markets.	

6.5.3. Strategy Whitelist

Introduced in	1.0
Last Revision	1.0
Package	Core
Source	https://github.com/OptimFinance/clean-code
Audits	https://github.com/OptimFinance/clean-code
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH

A whitelist, governed by ODAO, that holds ScriptHashes of strategies that can be created.

6.6. Stake Auction AMO

Introduced in	1.0
Last Revision	1.0
Package	Base
Source	Not Open Sourced
Audits	MLabs
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH

Stake Auction AMO is the second most important AMO of the system. It handles all of the system's staking operations operating in four steps:

1. Throughout the epoch anyone can create a bid, an offer of ADA or OADA payment for the system's staking power – For a single epoch snapshot.
 - If bids made totalling at least 250k ADA crosses the Clearing Rate, it will be matched then and there, without waiting until the epoch's end.
2. One hour before the epoch ends, no bids can be canceled or made. The system undeploys ADA out of other strategies and deploys it into the Staking Auction AMO to fill bids from the highest APY offered to the lowest until it can match no more. Bids made with OADA are treated as being 0.2% higher and the OADA is immediately burned upon being matched.
3. Bids filled in the previous step sit in a Lock, staked with the bid owner's key, until the next epoch one hours later. They are then unlocked and all of the ADA gathered.
4. The ADA is redeployed into the other strategies for the duration of the epoch.

The System's Clearing Rate is defined as the following function:

$$BR + PR * \left(1 - \frac{1}{1 + \exp\left(-\frac{T-50}{100} * 2\right)}\right) * \begin{cases} \left(1 + \frac{\frac{SRIL}{SR} - \frac{T}{100}}{1 - \frac{T}{100}}\right)^{5.6 + 2.6 * \left(\frac{T}{100}\right)} & \text{if } \frac{T}{100} < \frac{SRIL}{SR} \\ 1 & \text{otherwise} \end{cases}$$

Where

- SR = System Reserves (in ADA, total)
- SRIL = System Reserves In Locks (already offloaded stake)
- PR = 2% = Projected Rate (What the system expects the opportunity cost of not matching bids to be)
- BR = 2.7% = Base Rate (What is the bid floor, set close to the staking rate on ADA)
- T = Time in epoch, a number from 0 to 100 uniformly dividing the entire cardano epoch

The Clearing Rate is enforced by the Controller, and can be changed without updating the smart contract and will be updated by it with oversight and approval of the ODAO Council based on the current state of the market and analysis performed.

It's worth noting that the formula is subject to the dynamic inflow and outflow of reserves from and out of the system.

6.6.1. Stake Auction Bid

Introduced in	1.0
Last Revision	1.0
Package	Base
Source	Not Open Sourced
Audits	MLabs
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
The Bid is a simple script that enables the user-facing functionality of the Stake Auction AMO. It has an owner that can cancel it anytime except in the last 1 hour of the epoch. And it can be filled by a Stake Auction AMO, ensuring that the owner receives the stake power.	

6.6.2. Stake Auction Lock

Introduced in	1.0
Last Revision	1.0
Package	Base
Source	Not Open Sourced
Audits	MLabs
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
An even simpler script that tracks the epoch it was locked in, allowing unlock to the Stake Auction AMO that matched the bid in the next epoch.	

6.6.3. OADA Stake Auction Rule

Introduced in	1.0
Last Revision	1.0
Package	Base
Source	Not Open Sourced
Audits	MLabs
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
A yet even simpler simpler script that allows OADA received from bids into the Stake Auction AMO to be burned.	

6.7. Splash DEX AMO

Introduced in	1.0
Last Revision	1.0
Package	Base
Source	Not Open Sourced
Audits	MLabs
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
Quite possibly the most complex script in the system. A good joke here would be to not elaborate at all, but – What the system can do is broken up in 4 actions. 1. Protect Bottom Peg • The system can buy back OADA as long as the amount bought back is $\geq$ amount spent. • It needs liquidity sidelined in order to function. If the system runs out of idle capital, it either removes part of the supplied liquidity, or pulls it out of other strategies. • This functionality is invoked in two cases 1. Protecting the Bottom Peg 2. Rebalancing the pool for the addition of liquidity	

In the first case the controller chooses to protect the downside of the asset lower than the parameter may lead to believe alone, as discussed in an earlier section.

Currently via the first action the pool gets rebalanced up to price of **0.99** ADA per OADA, and via the second, **0.999**. The second of those two numbers is not 1:1 because of an implied 0.1% swap fee.

The difference between OADA received (and burnt) and ADA sent is profit for the system.

2. Protect Upper Peg

- The system can sell freshly minted OADA as long as the amount of ADA coming in $\geq$ amount minted. The difference between those two is the system profit. This can be thought of as a round-about way to mint OADA, equivalent to using the Deposit AMO
- Does not have any need for idle liquidity, and works by invoking the DEX Rule.

3. Add Liquidity

- Uses reserve ADA paired with freely minted OADA via the DEX Rule to supply liquidity.
- Can only perform the action (and mint the OADA to pair) if the price on the open market is at least **0.999** ADA per OADA, and no more than **1.001** ADA per OADA. This, counting in a 0.1% swap fee, ensures that none of this minted OADA can be issued (leave the LP) for a price of less than 1 ADA – keeping the token 1:1 backed. For a more formal proof of system property, refer to a later section.
- The above assurance holds regardless of swaps, deposits and withdrawals into and out of the pool after the system supplied the liquidity, as trivially seen.
- Records the sum of the OADA and ADA supplied to record the profit upon withdrawal

4. Remove Liquidity

- Pulls OADA+ADA tokens out of the pool, and, in proportion to the withdrawn LP it holds, takes account of the total sum received (i.e. if it holds 100 LP and supplied 1000 ADA + OADA, then the withdrawal of 50 LP that gets the system 600 ADA+OADA implies the system has made a 100 ADA profit)
- Immediately burns the OADA pulled out of the pool.

Note: Given mathematical perfection is not possible with floating point arithmetic and a negligible but non-zero splash protocol fee – some error tolerance must be had, and the system is set to have a *slightly tighter* range for deposits. Therefore there is a range of current price to target price that is impossible to rebalance to without the system taking on a loss for doing so. It is incredibly precise and unreasonable to expect to be able to be induced on the open market for long, but it can cause the system to be unable to perform the add liquidity action temporarily. This is expected behavior, and the OADA controller simply waits until the price moves to a price from which it can rebalance the price higher without taking on a loss. (i.e. system has to profit enough by repegging it from slightly lower to push it slightly into territory where it loses money by pushing the open market price into – as it cannot make a losing trade)

6.7.1. OADA Splash DEX Rule

Introduced in	1.0
Last Revision	1.0
Package	Base

Source	Not Open Sourced
Audits	MLabs
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
As the DEX AMO is most complex, this is in turn, as one could expect, the most complex rule, one that enables each of the 4 actions that must be performed.	

6.8. Donation Strategy

Introduced in	1.0
Last Revision	1.0
Package	Core
Source	https://github.com/OptimFinance/clean-code
Audits	https://github.com/OptimFinance/clean-code
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
The only Strategy in the system, its purpose is to allow anyone to donate ADA to the system to count it as profit. Mostly used for working around Catalyst, but also given as the simplest example of a strategy. A note on the terminology used, as it doesn't have an associated Rule, it is considered a Strategy. If it were to be enriched with a Rule, to burn donated OADA (of which donations are currently unaccepted) then it would be considered an AMO.	

6.9. FluidV3-Lending AMO

Introduced in	2.0
Last Revision	2.0
Package	Forward
Source	Not Open Sourced
Audits	None Currently
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
This AMO allows the second primary way of minting OADA, as opposed to swapping 1:1 for ADA, it issues OADA against non-ADA collateral assets as credit utilizing the FluidTokens V3 contracts. The entire module boils down to accounting for the profits from lending against collateral defined in a configuration utxo. Each of these contains the maximum duration that can be lent out, the LTV at liquidation and origination, oracle that defines mark price of the asset, as well as maximum market size. Additionally, it contains the liquidation strategy, which allows for arbitrary logic of liquidation of defi-integrated or simple assets.	

6.9.1. FluidV3-Lending Simple Liquidation Strategy

Introduced in	2.0
Last Revision	2.0
Package	Forward
Source	Not Open Sourced
Audits	None Currently
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
The simple liquidation strategy is intended to be used with simple assets that trade directly with good volumes.	

The liquidation strategy simply takes all of the collateral upon liquidation trigger (time, or price based) and immediately moves it into the Liquidation Auction, with the starting price provided by the oracle.

Furthermore the Liquidator does accounting for PNL of the liquidation and reports it to the Strategy for further accounting.

6.9.2. FluidV3-Lending Liquidation Auction

Introduced in	2.0
Last Revision	2.0
Package	Forward
Source	Not Open Sourced
Audits	None Currently
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
Auction script sells directly into ADA and OADA paired pools. Auction does not have limits on which DEX, or liquidity source it takes liquidity from. Currently offchain is configured to utilize OADA and ADA paired pools (royalty and simple) on Splash. Price starts at oracle price, and decays using TICK_LENGTH, TICK_AMOUNT, and TICK_DECAY parameters. By default and unless otherwise specified, the parameters are: TICK_LENGTH=10m, TICK_AMOUNT=144, TICK_DECAY=0.03 Which means, each 10 minute interval, the minimum price the auction wants to sell at, is decreased by 3%. Potentially liquidating for the entire duration of the day (24 hours), but realistically finishing before the first hour ends.	

6.9.3. FluidV3-Lending Rule

Introduced in	2.0
Last Revision	2.0
Package	Forward
Source	Not Open Sourced
Audits	None Currently

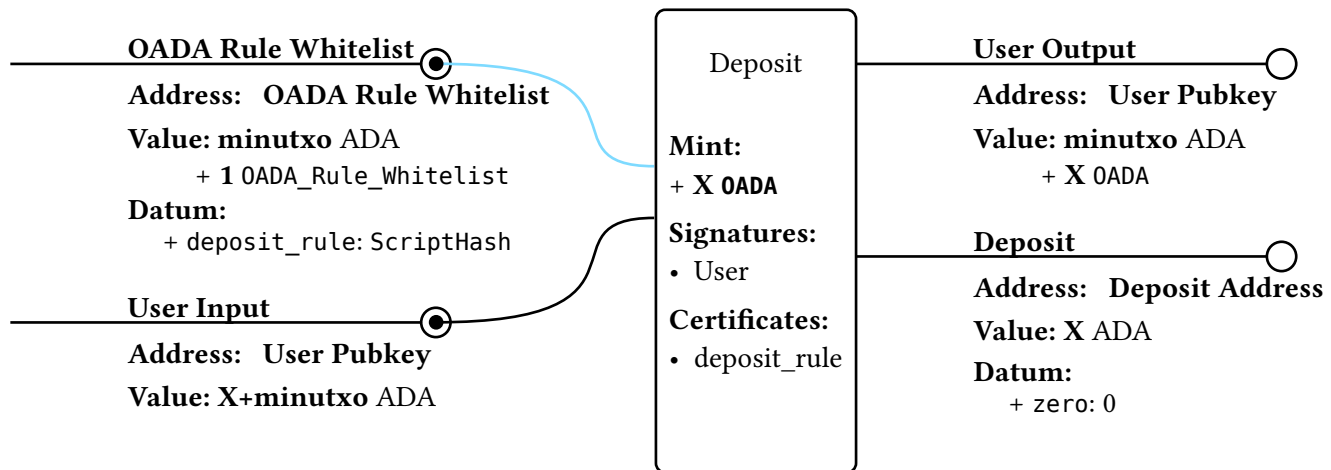
Plutus Version	2
Unparameterized Script Hash	HASH
Deployed Script Hash	HASH
The AMO must be present as input for it to mint. Three conditions, deferred to the AMO logic to verify: Create Pool, Cancel Pool, Burn Repayment.	

7. System Transactions

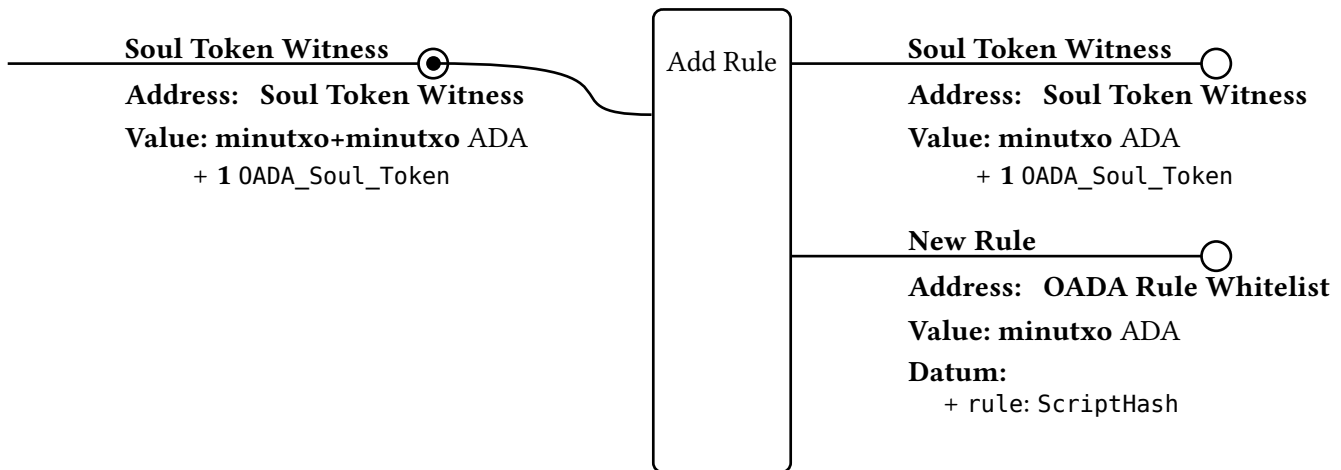
In this section we present the totality of the transaction schemas used in the system's operation. We present the *minimal* version of each transaction, as we attempted to make it possible to include other contracts and transfers while performing the system operations – nevertheless do verify in code before assuming that one's use case is permitted to be composable with a given system.

Some liberties are taken in order to present the system on a conceptually coherent level, and simplifications made. This is a catalogue of all of the actions performed by the system and not a full specification of them.

7.1. Mint OADA

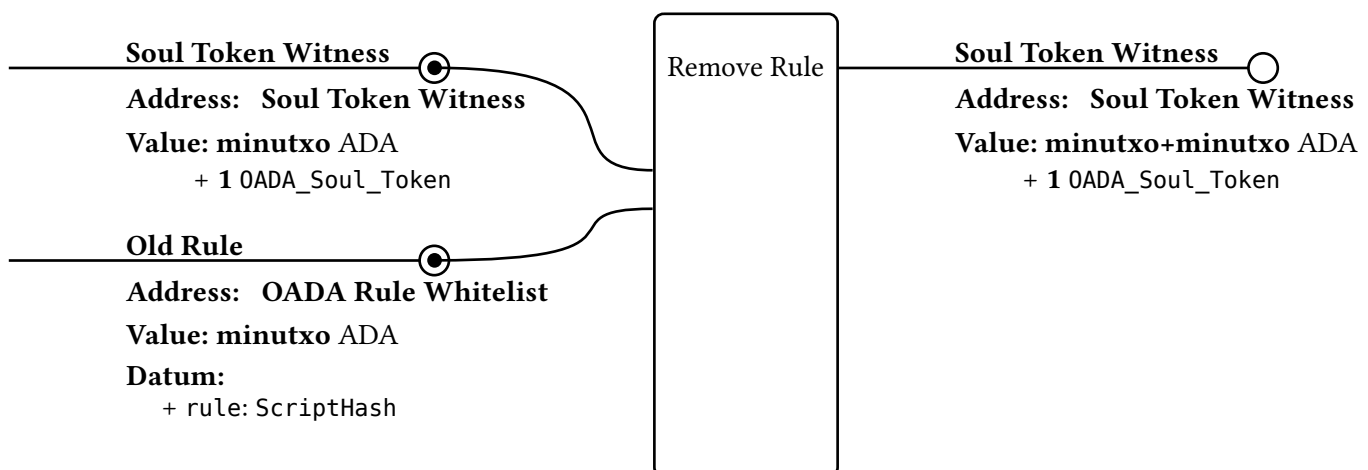


7.2. Add OADA Rule



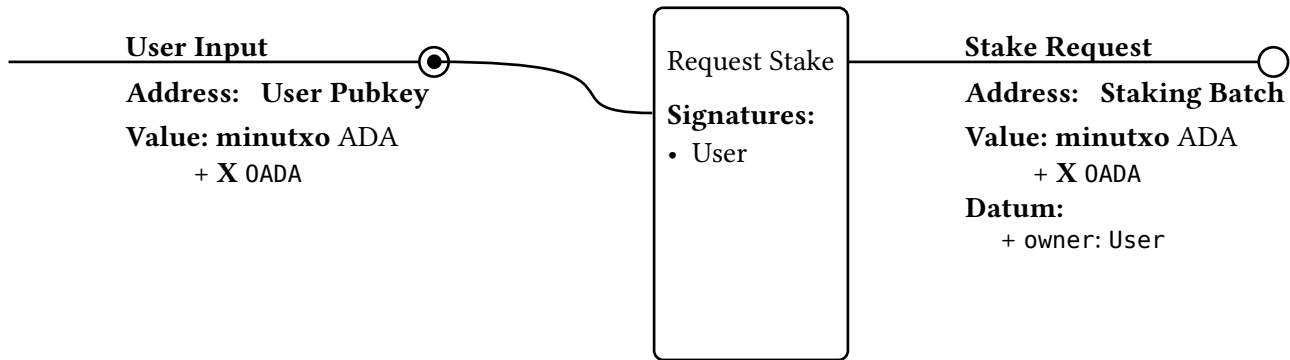
Note: Script is inlined

7.3. Remove OADA Rule

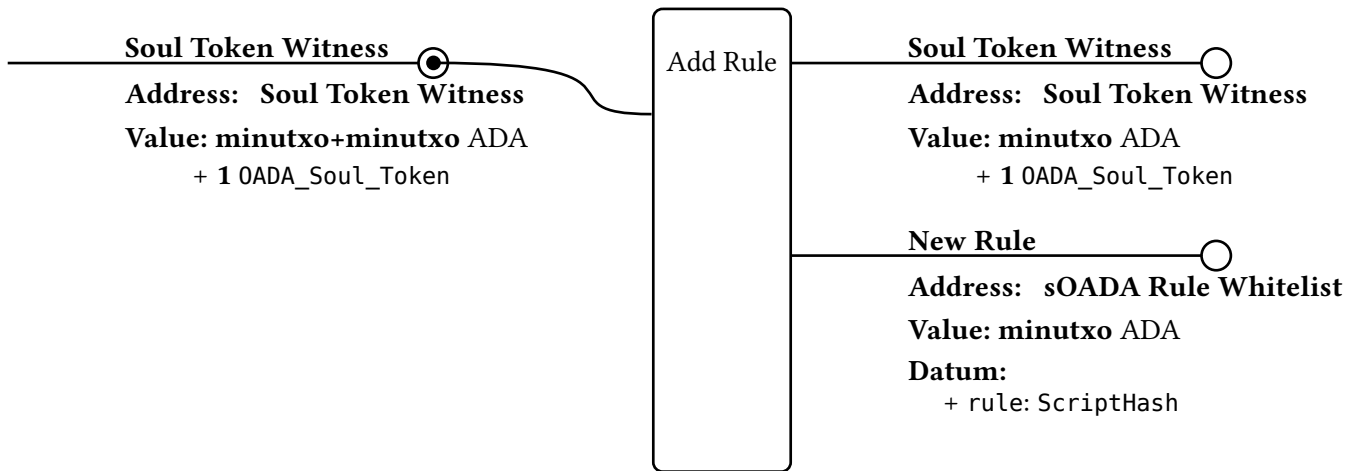


Note: Script is inlined

7.4. Request Stake of OADA

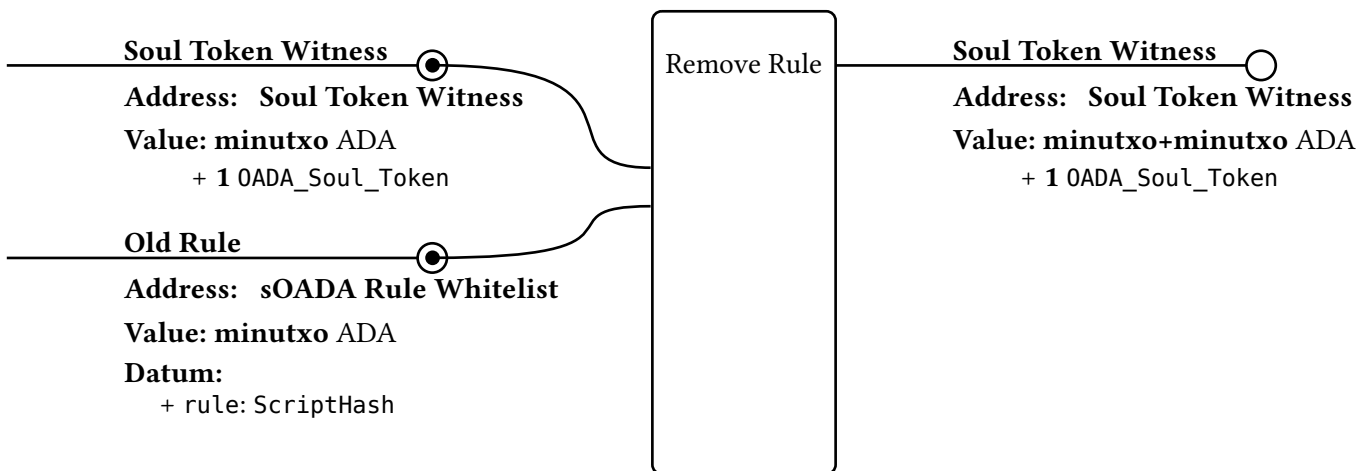


7.5. Add sOADA Rule



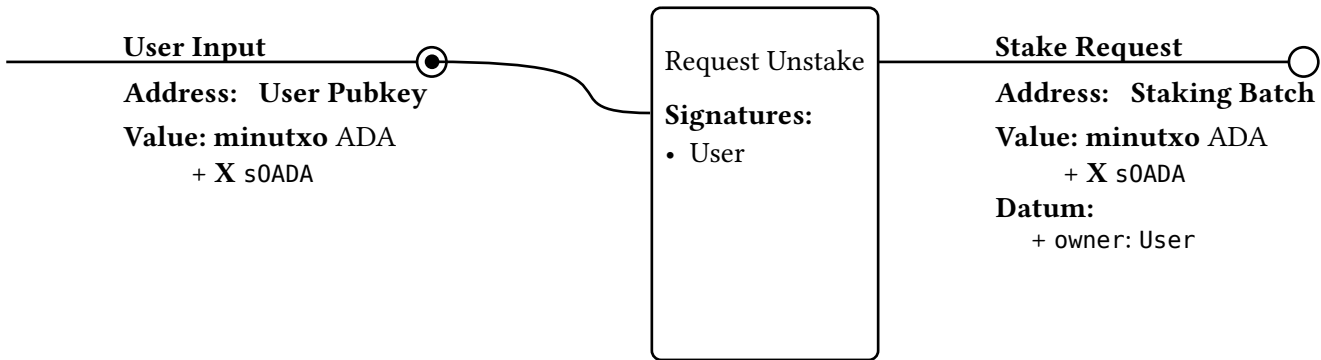
Note: Script is inlined

7.6. Remove sOADA Rule

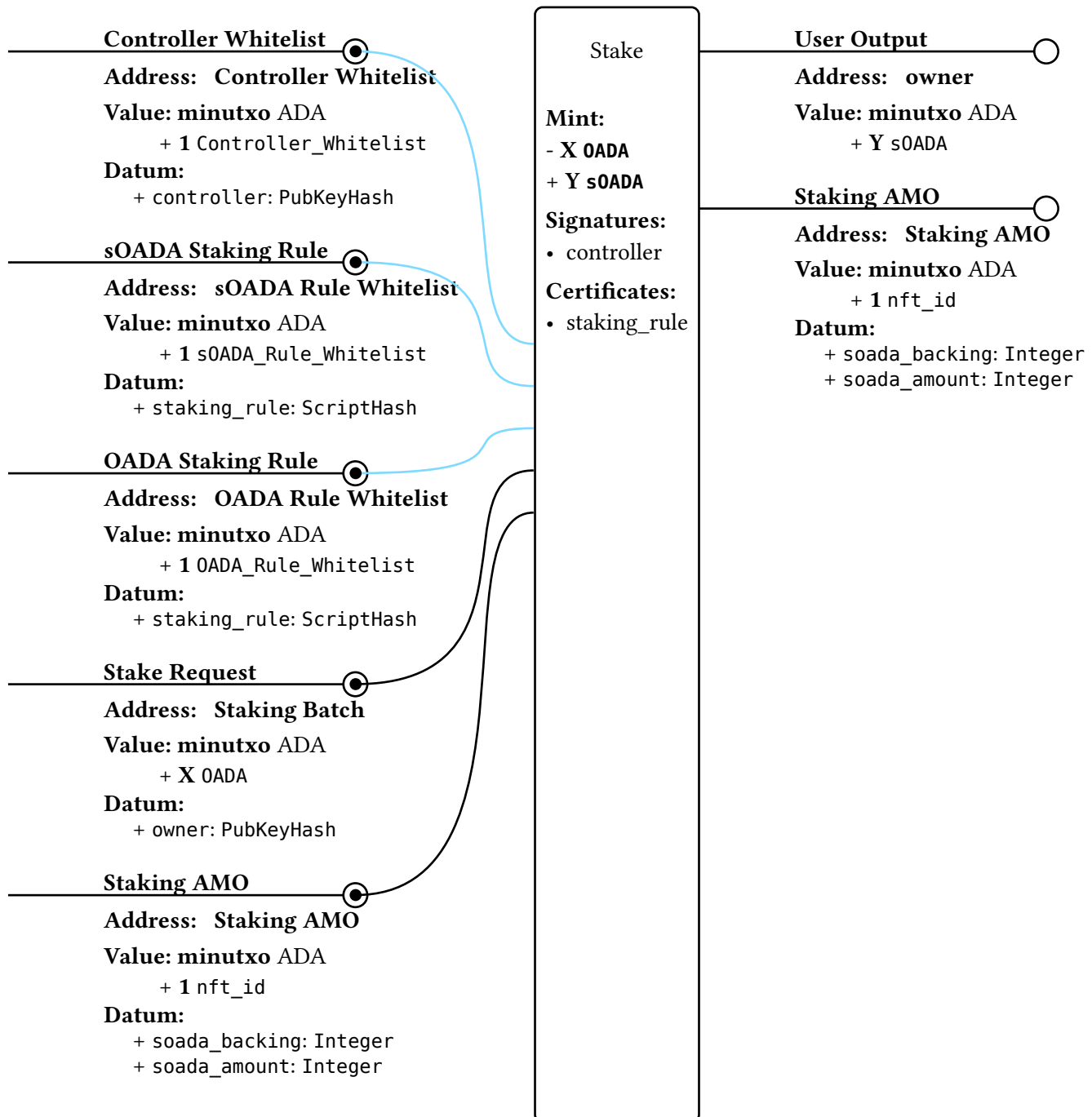


Note: Script is inlined

7.7. Request Unstake of sOADA



7.8. Stake OADA for sOADA



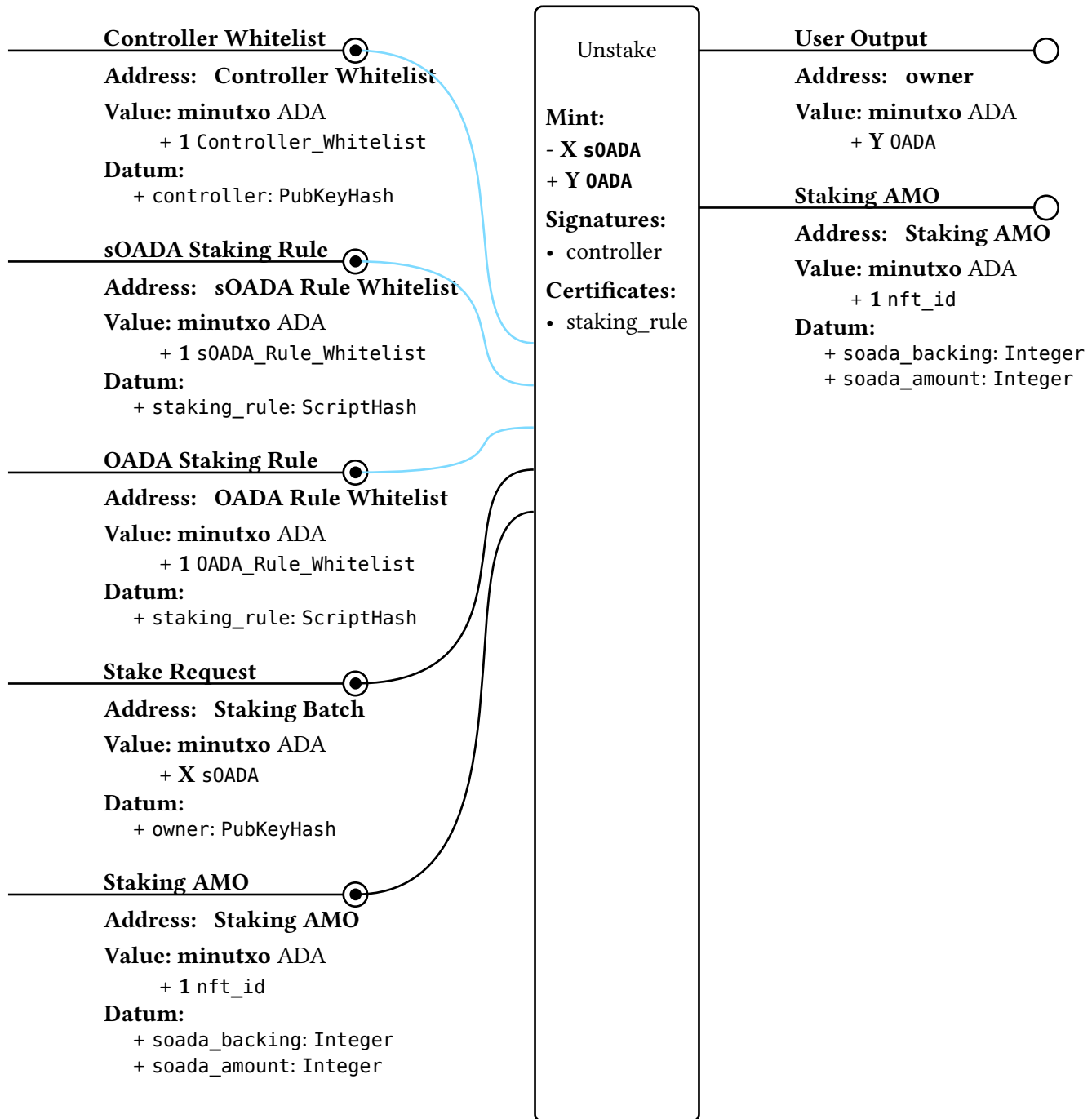
Note:

$$Y = X * \text{sOADA_backing} / \text{sOADA_amount}$$

Updates the 'soada_backing' and 'soada_amount' to reflect the action

The 'staking_rule' is the same script for OADA and sOADA

7.9. Unstake sOADA for OADA



Note:

$$Y = X * 0.999 * \text{sOADA_amount} / \text{sOADA_backing}$$

Updates the 'soada_backing' and 'soada_amount' to reflect the action

The 'staking_rule' is the same script for OADA and sOADA

7.10. Perform Staking AMO Parameter Update

Soul Token Witness

Address: Soul Token Witness

Value: minutxo ADA
+ 1 OADA_Soul_Token

Staking AMO

Address: Staking AMO

Value: minutxo ADA
+ 1 nft_id

Datum:

+ odao_fee: Integer
+ soada_limit: Integer

Update Parameters

Soul Token Witness

Address: Soul Token Witness

Value: minutxo ADA
+ 1 OADA_Soul_Token

Staking AMO

Address: sOADA Rule Whitelist

Value: minutxo ADA
+ 1 nft_id

Datum:

+ odao_fee: Integer
+ soada_limit: Integer

7.11. Claim ODAO Fees

ODAO Rule Whitelist

Address: ODAO Rule Whitelist

Value: minutxo ADA
+ 1 OADA_Rule_Whitelist

Datum:

+ fee_claim_rule: ScriptHash

ODAO Fee Claimer

Address: ODAO Fee Claimer

Value: minutxo ADA
+ 1 OADA_Fee_Token

Staking AMO

Address: Staking AMO

Value: minutxo ADA
+ 1 nft_id

Datum:

+ odao_soada: Integer
+ soada_backing: Integer
+ soada_amount: Integer

Claim Fees

Mint:

+ X OADA

Signatures:

•

Certificates:

• fee_claim_rule

ODAO Fee Claimer

Address: ODAO Fee Claimer

Value: minutxo ADA
+ 1 OADA_Fee_Token
+ X OADA

Staking AMO

Address: sOADA Rule Whitelist

Value: minutxo ADA
+ 1 nft_id

Datum:

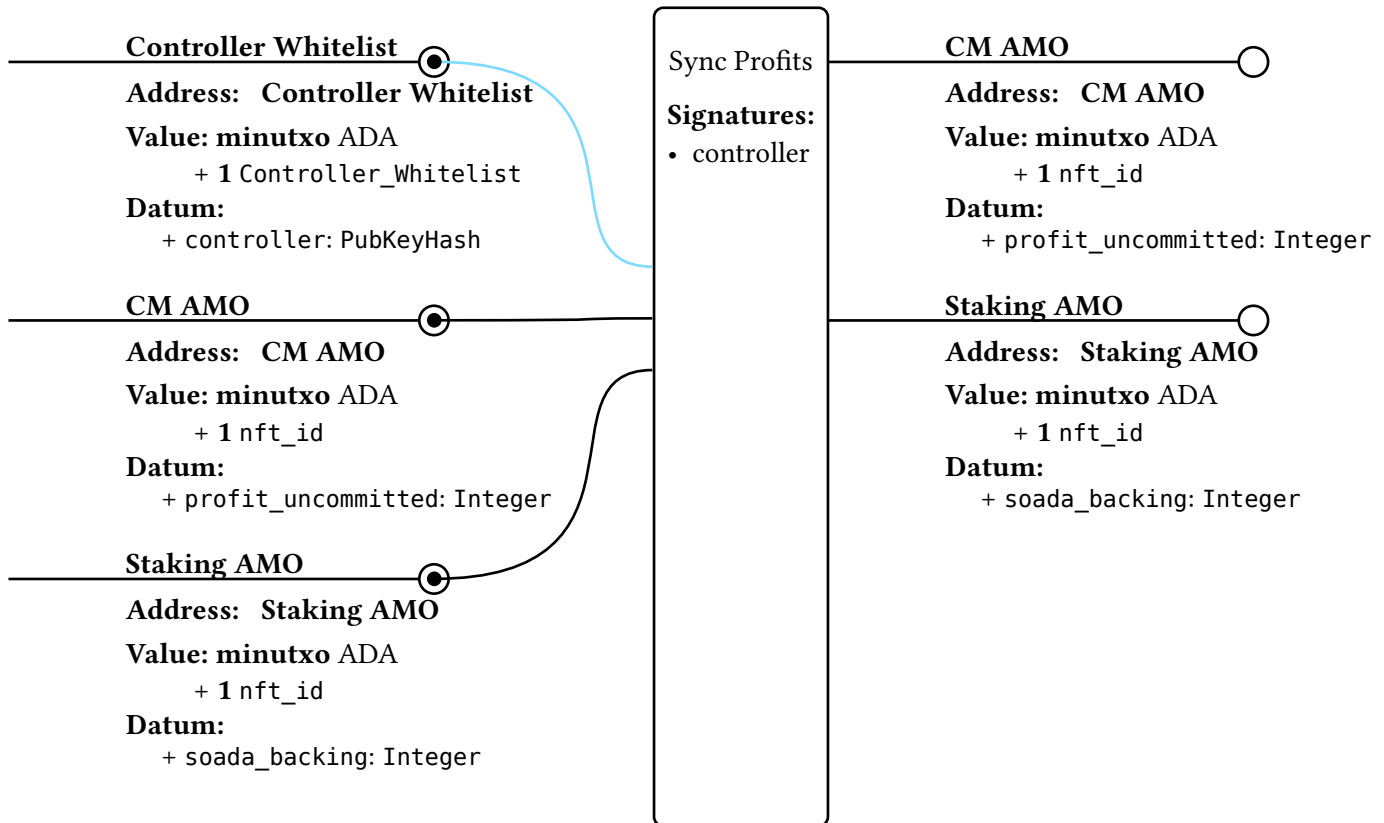
+ odao_soada: Integer
+ soada_backing: Integer
+ soada_amount: Integer

Note:

$$X = (\text{input 'odao_soada'} - \text{output 'odao_soada'}) * \text{soada_backing} / \text{soada_amount}$$

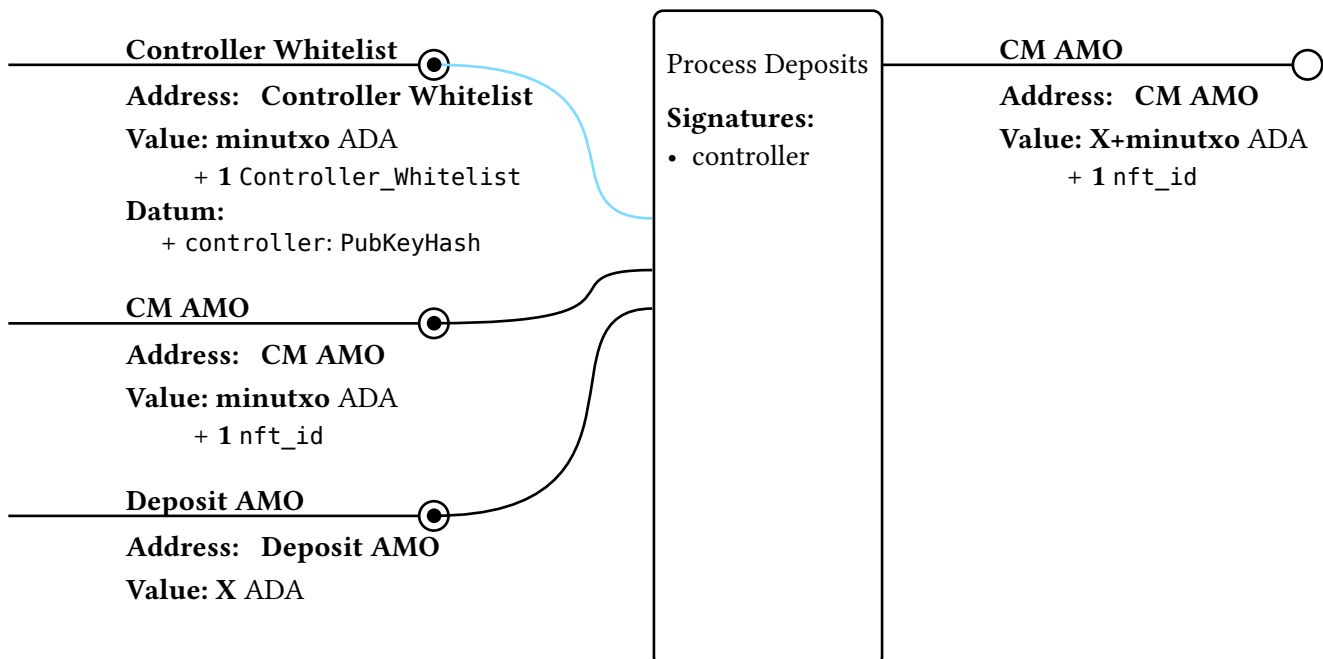
Importantly, upon fee claim, the odao skips the 0.1% unstake fee present on regular staking

7.12. Sync Profits with Staking AMO



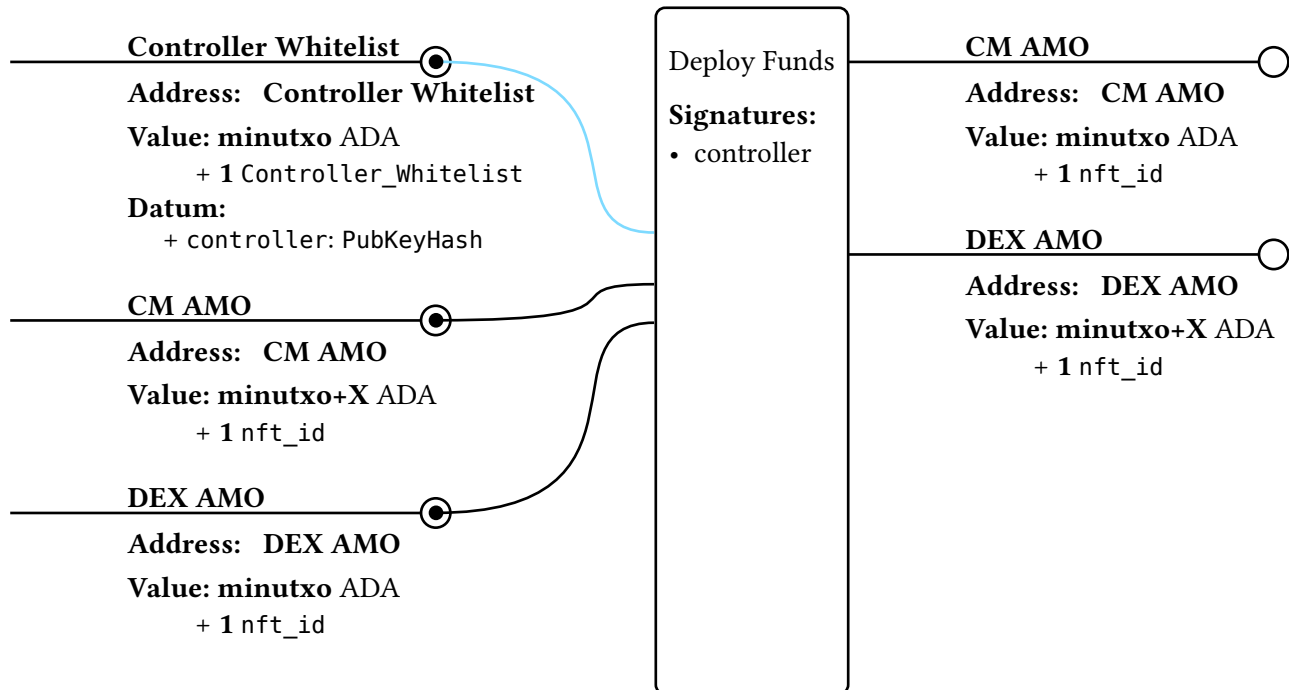
Note: profit_uncommitted is added to soada_backing

7.13. Process Deposits in Collateral Management AMO

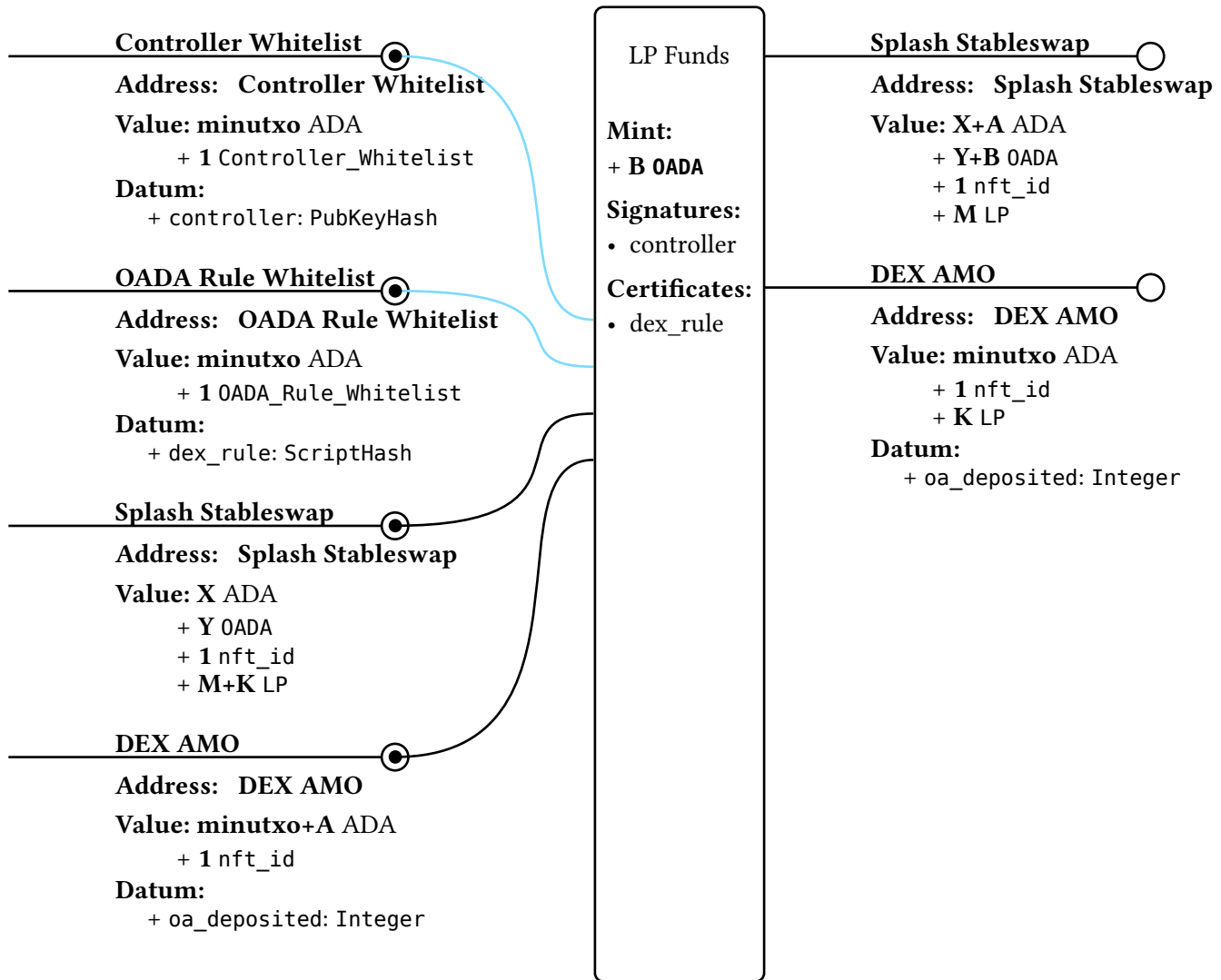


Note: Can process multiple dozen deposits at a time

7.14. Deploy Funds into the Splash DEX AMO



7.15. Deposit ADA with OADA into Splash Stableswap



Note:

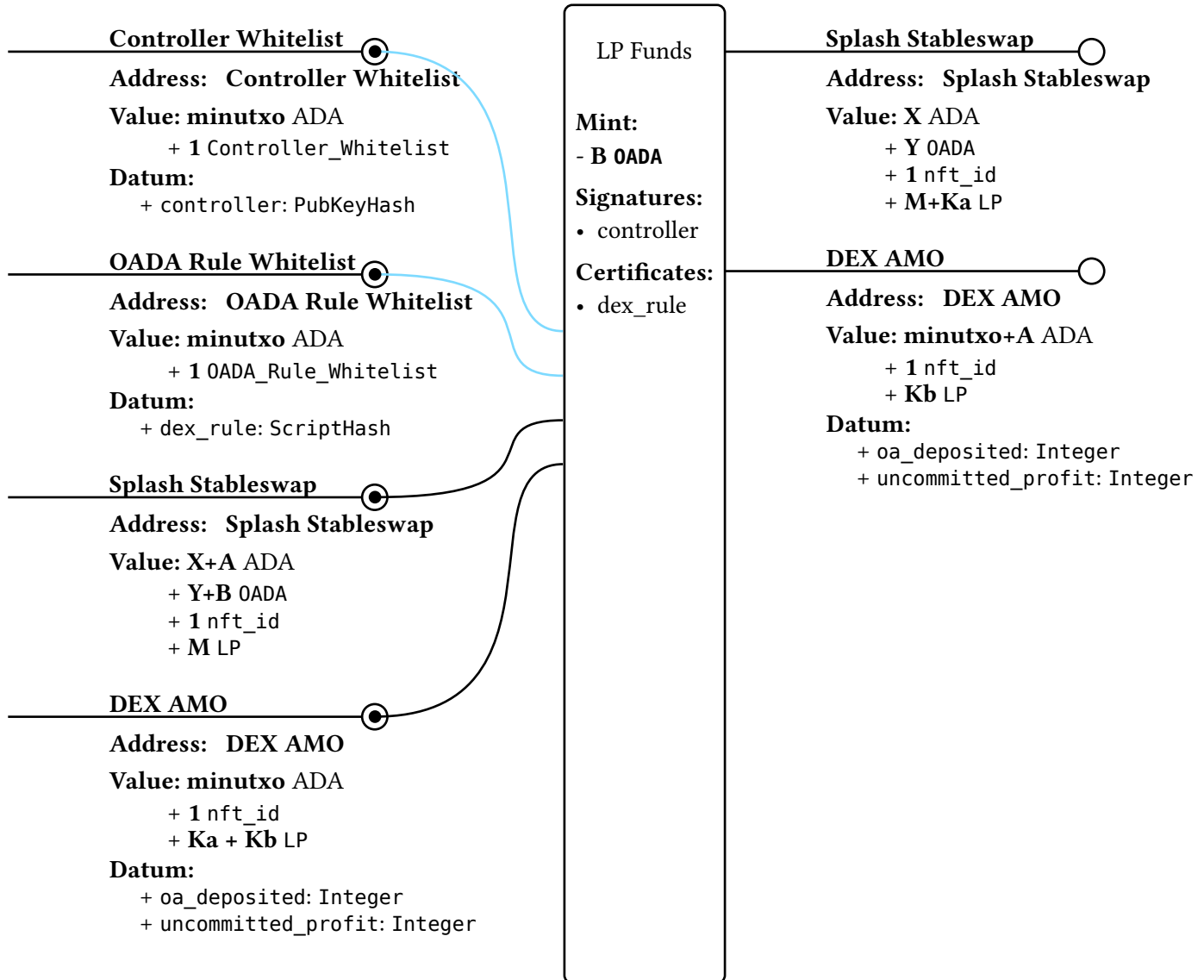
$\text{deposit_guard} \leq X / Y$

$\text{deposit_guard} \geq Y / X$

deposit guard ensures the deposit happens between 0.999 and 1.001

A+B is added to oa_deposited

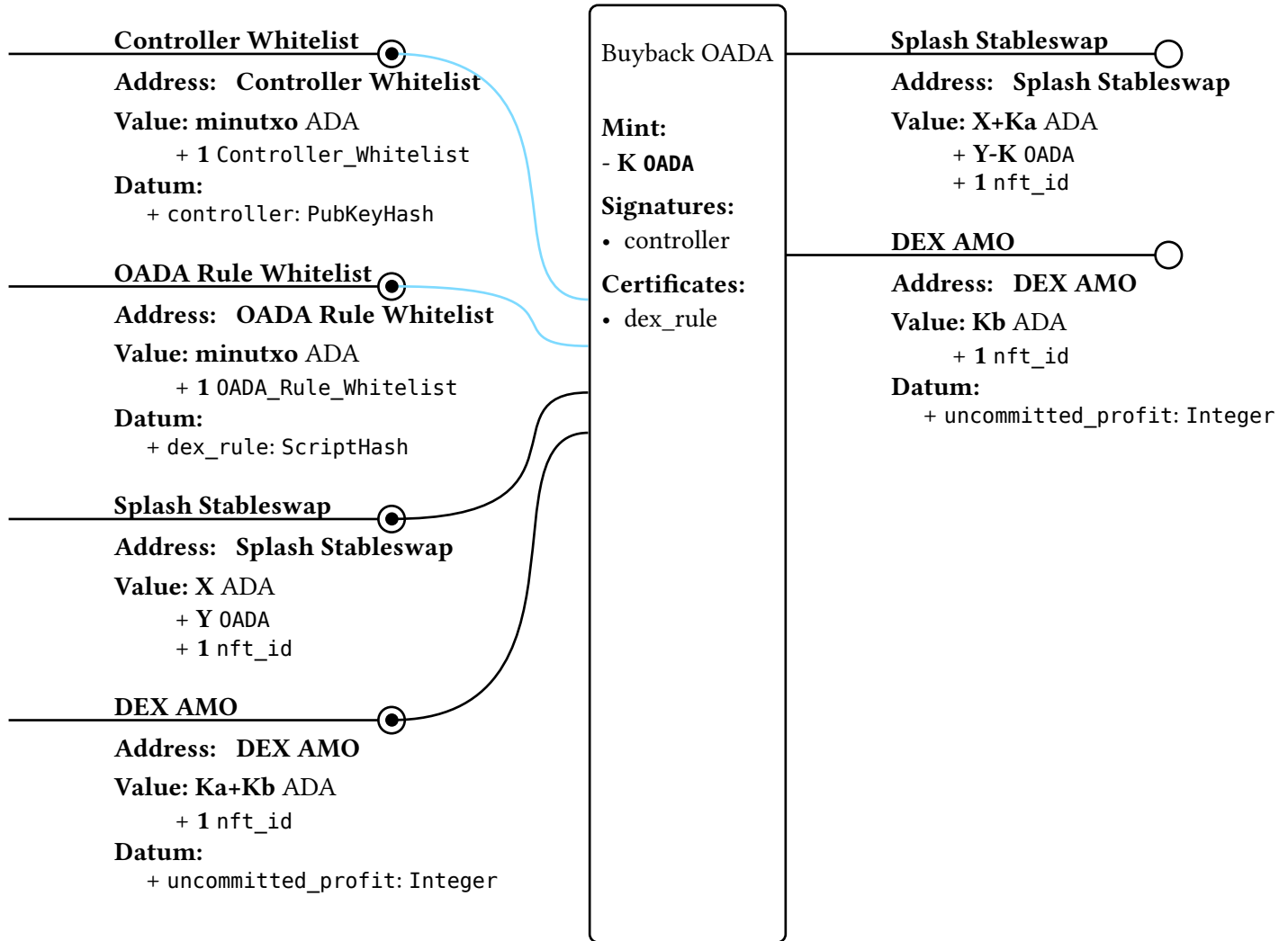
7.16. Withdraw LP from the Stableswap



Note:

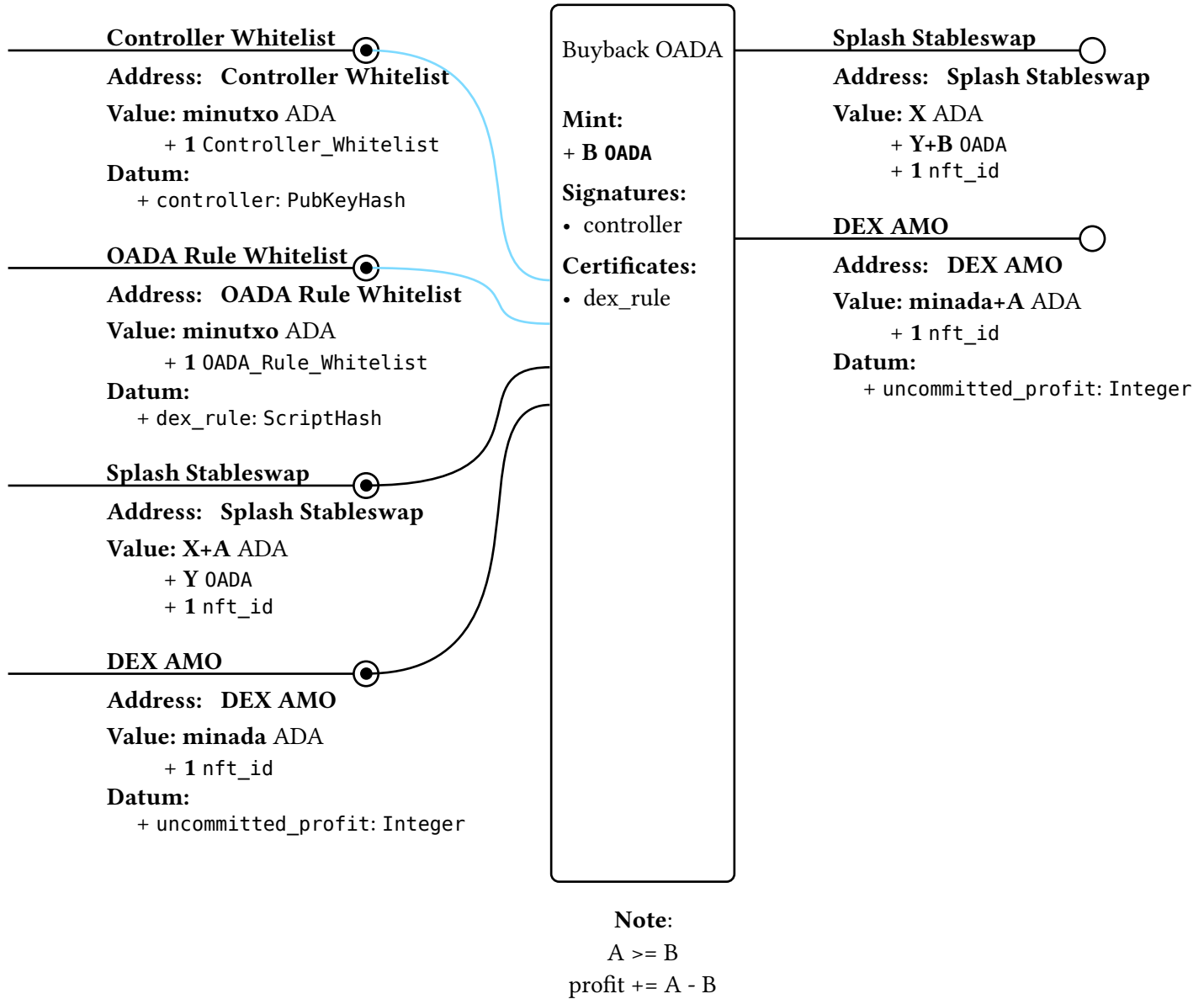
A+B is subtracted from oa_deposited
profit += (withdrawn - deposited) * (Ka / (Ka + Kb))

7.17. Protect Downside Peg on the open market

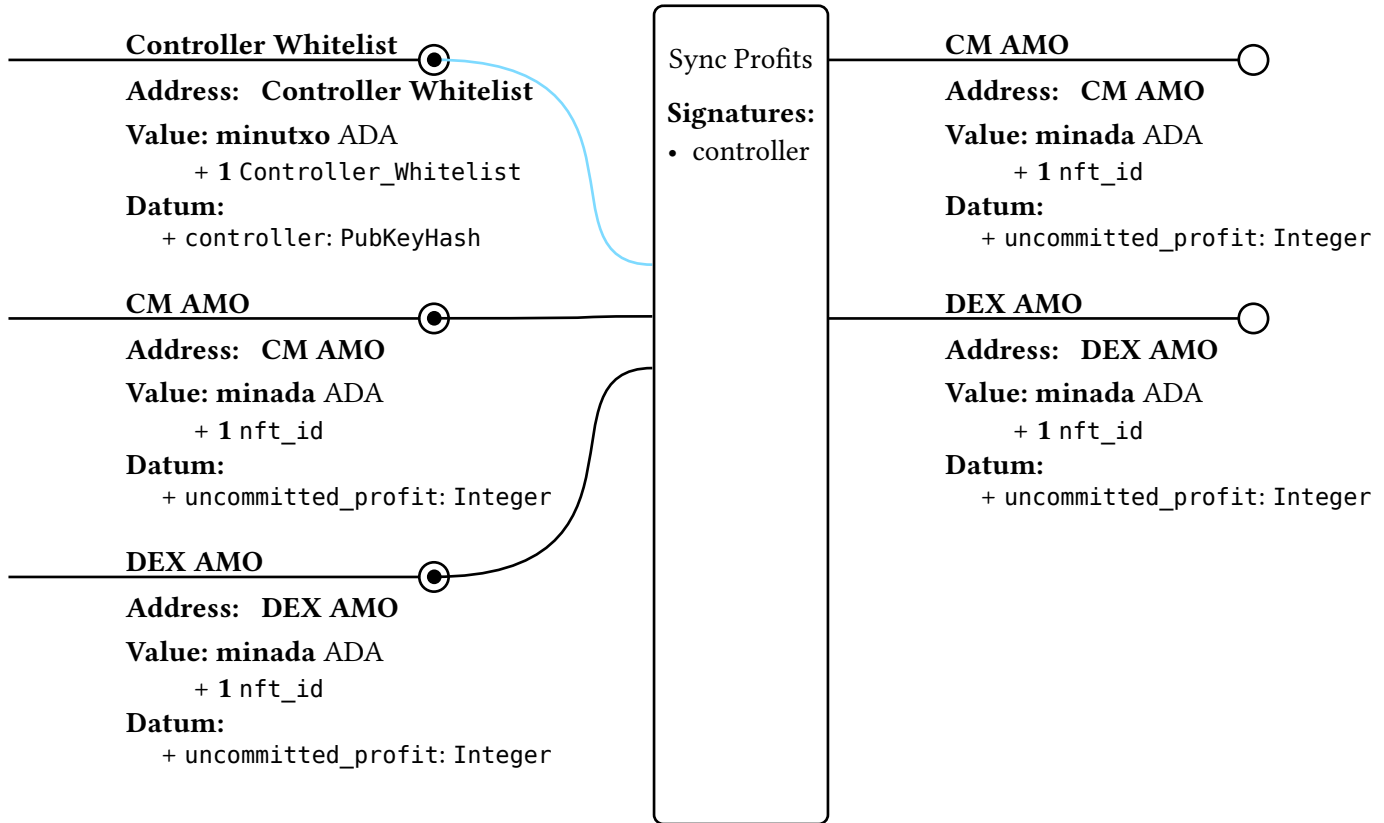


Note:
 $K \geq K_a$
profit += $K - K_a$

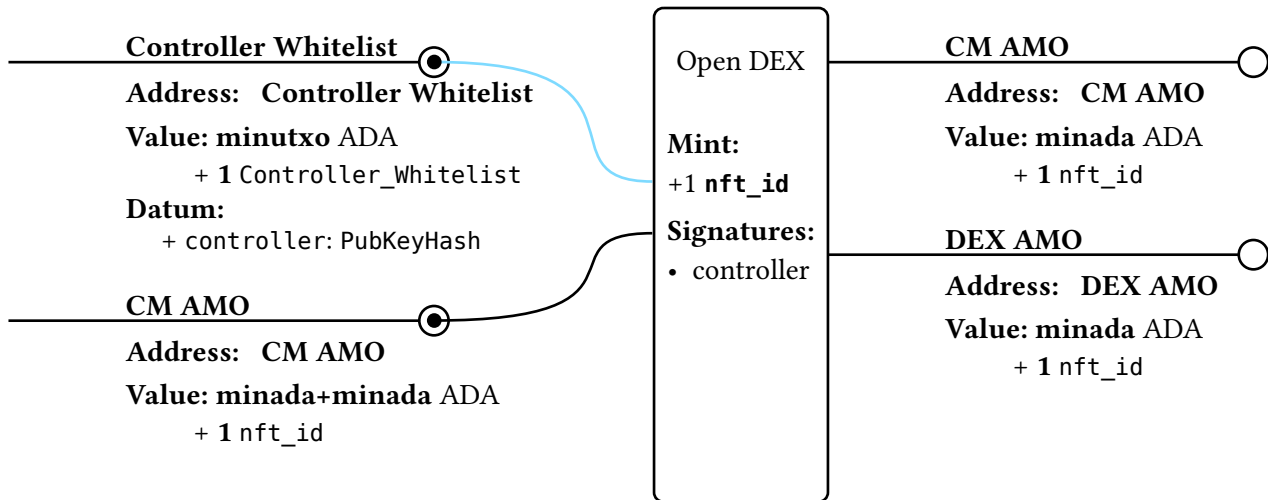
7.18. Protect Upside Peg on the open market



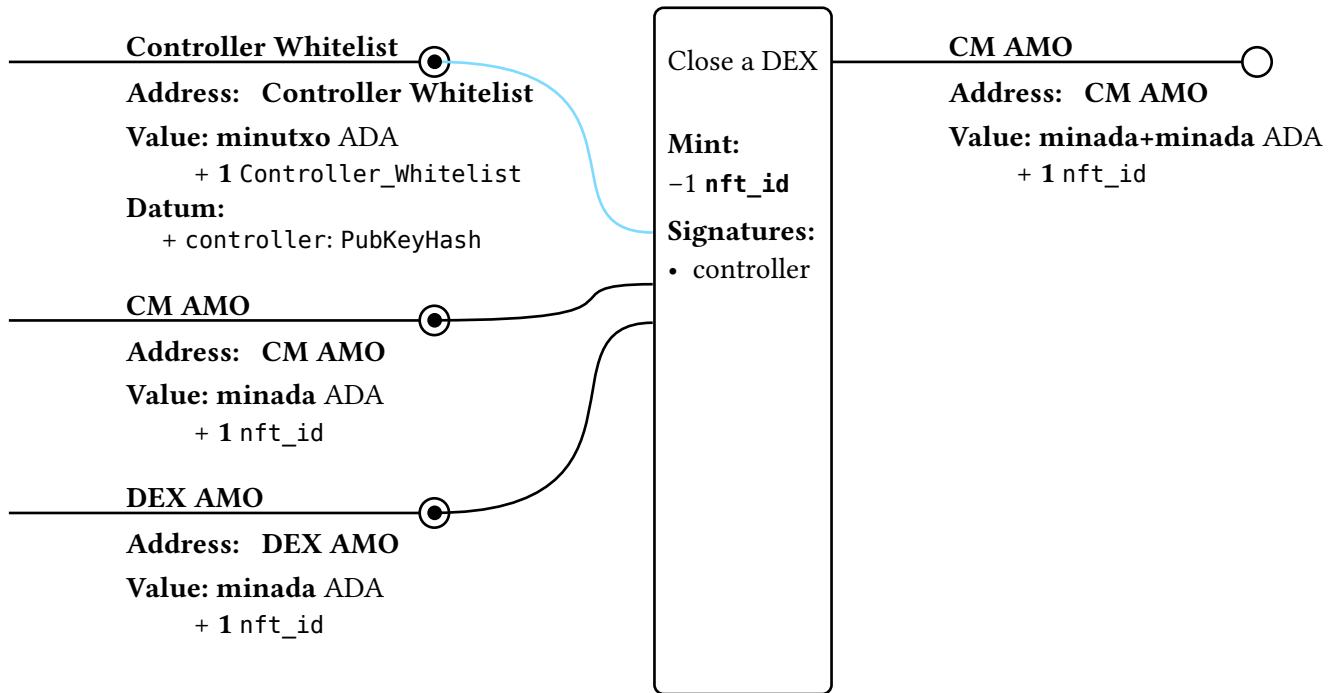
7.19. Sync DEX AMO Profits with the CM AMO



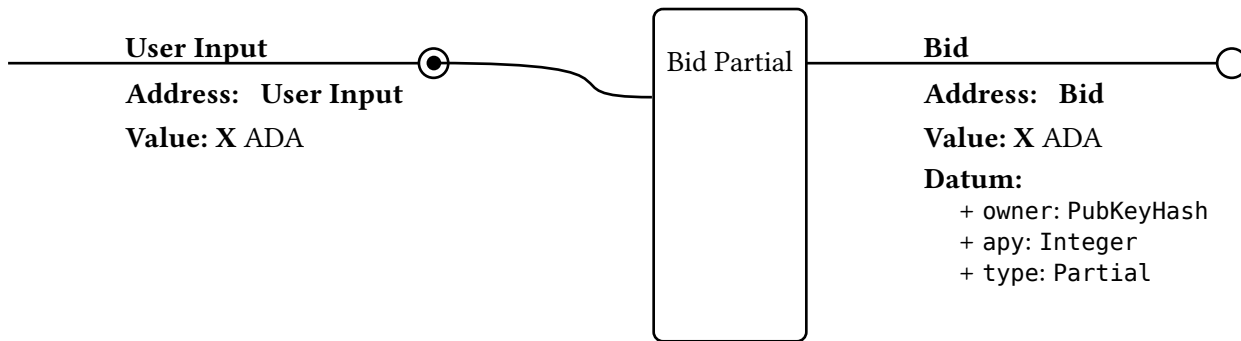
7.20. Open a DEX AMO



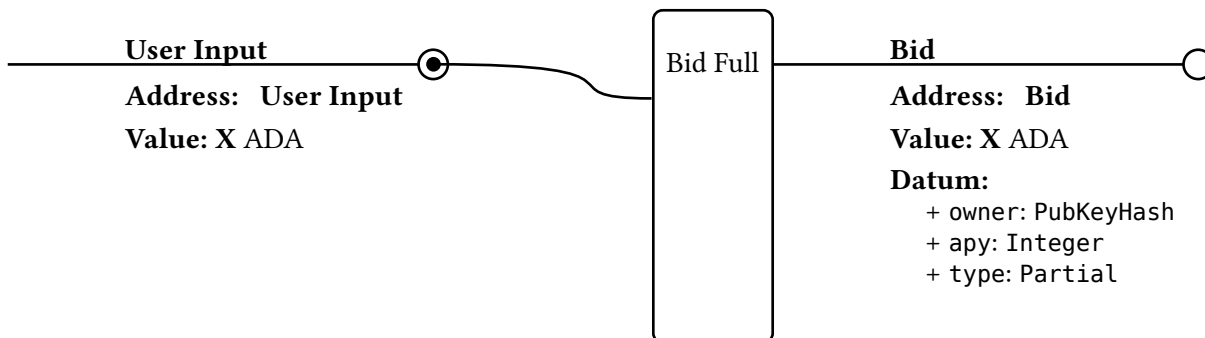
7.21. Close a DEX AMO



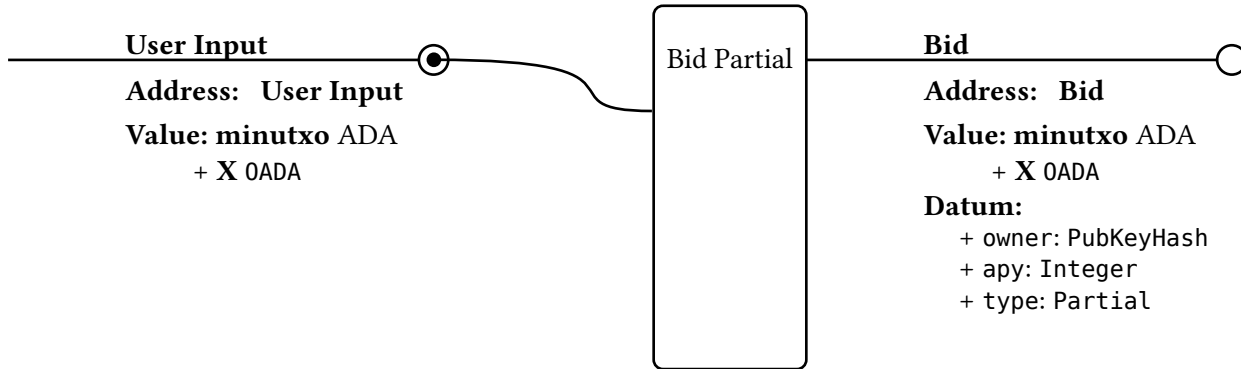
7.22. Partial Fill Bid ADA in the Staking Auction



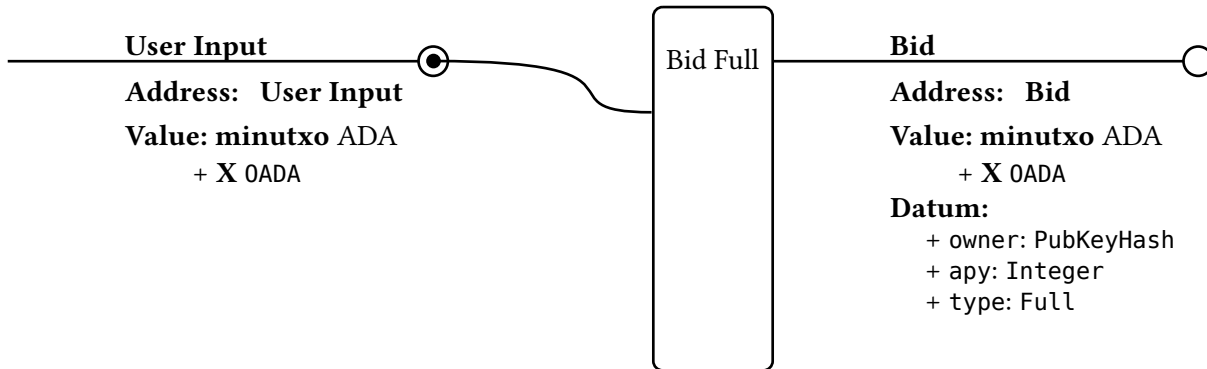
7.23. Fill or Kill Bid ADA in the Staking Auction



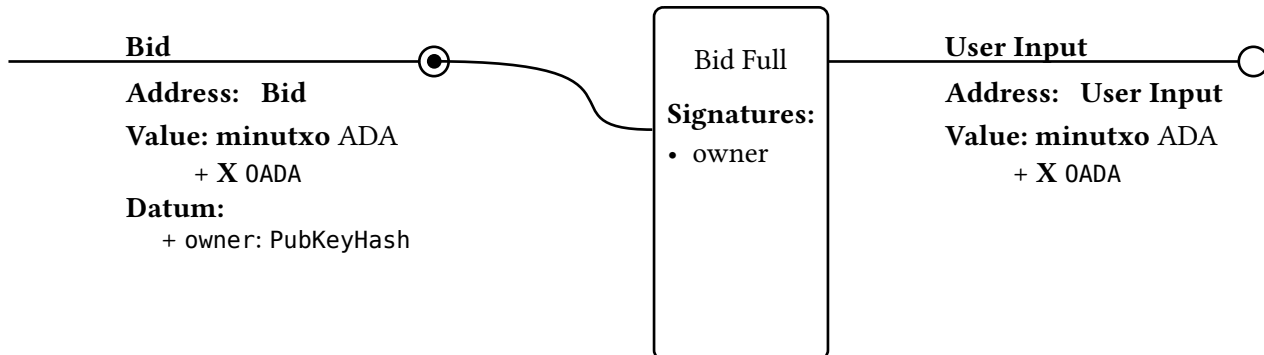
7.24. Partial Fill Bid OADA in the Staking Auction



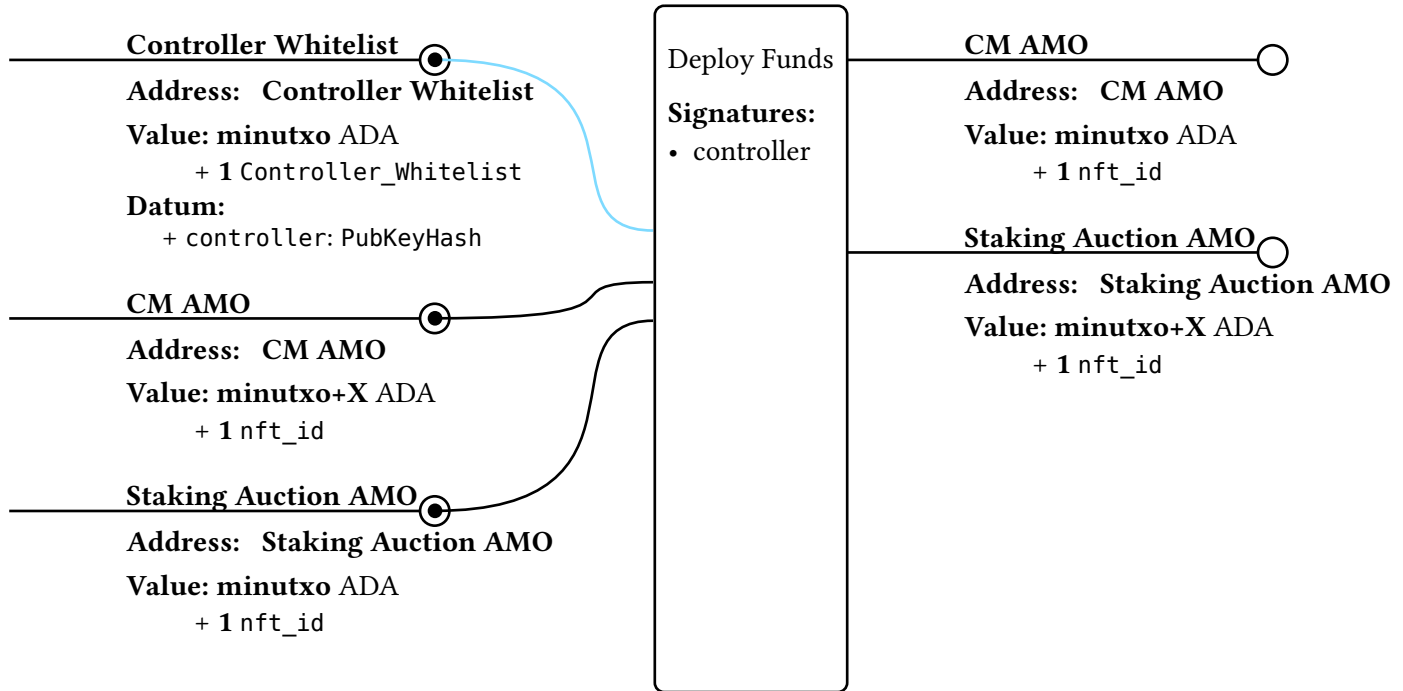
7.25. Fill or Kill Bid OADA in the Staking Auction



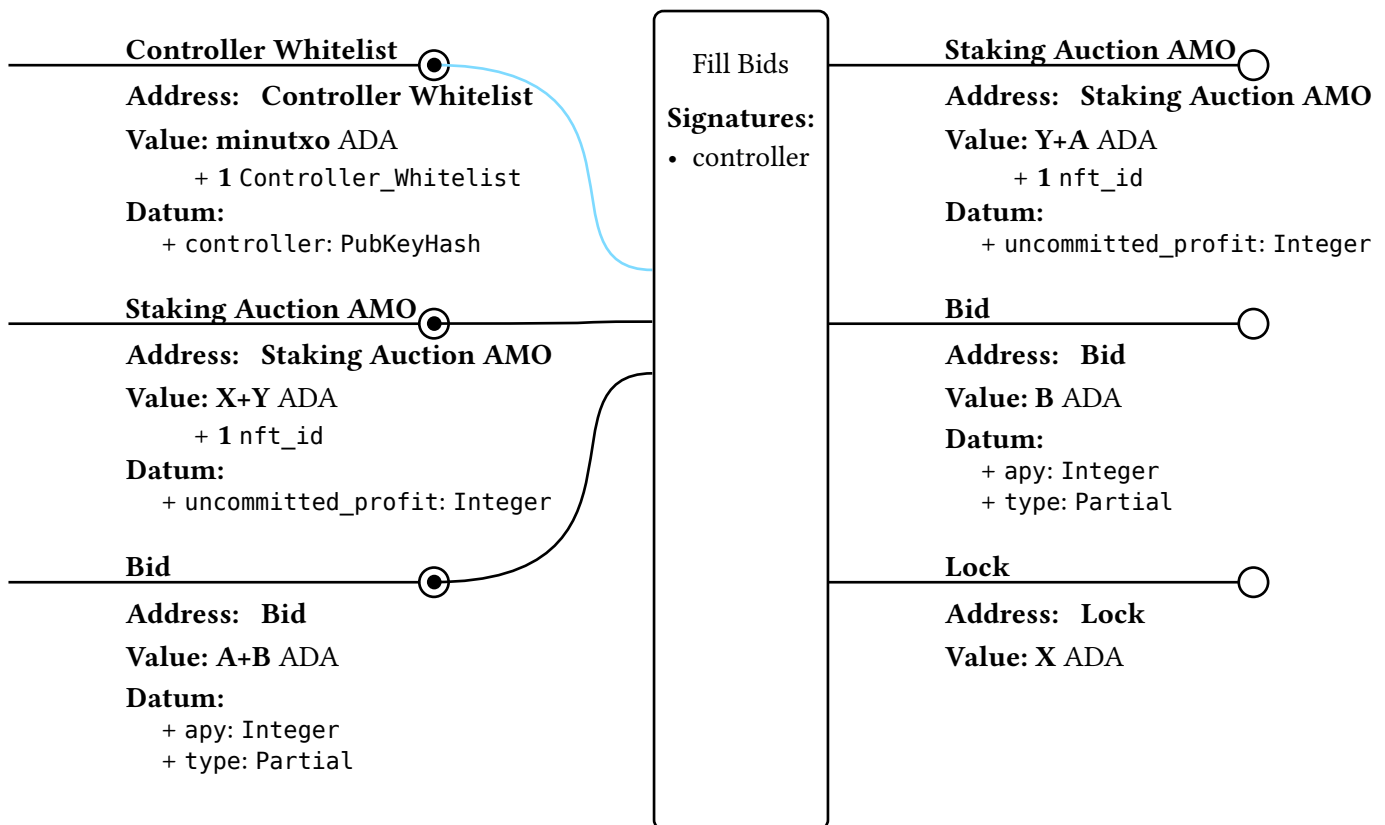
7.26. Cancel a bid



7.27. Deploy Funds into the Staking Auction AMO



7.28. Partial Fill a Partial Bid



Note:

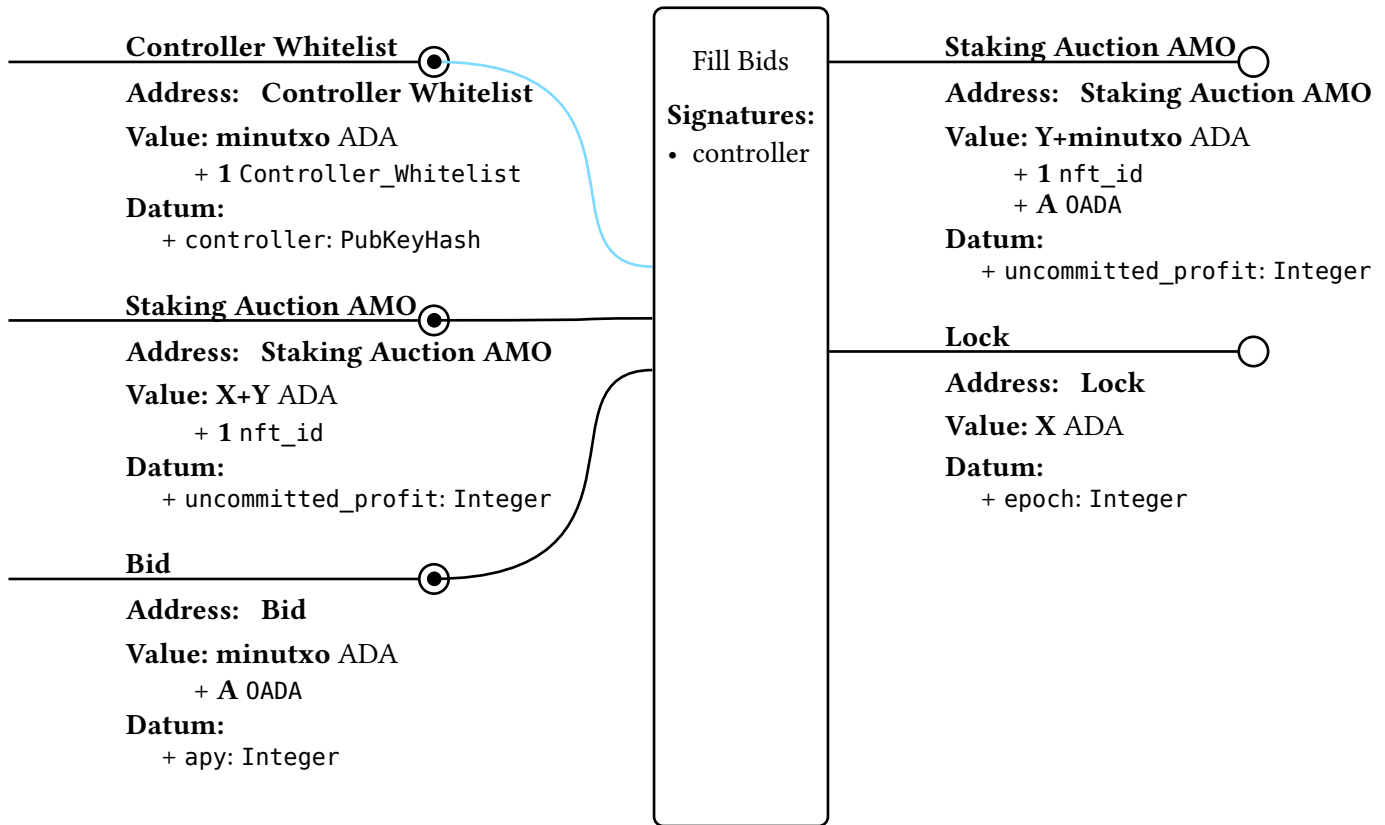
Can partial/fill multiple bids at once

$$X = B / (apy/73)$$

Lock has the current epoch in datum

uncommitted_profit += A

7.29. Fill a Fill or Kill Bid



Note:

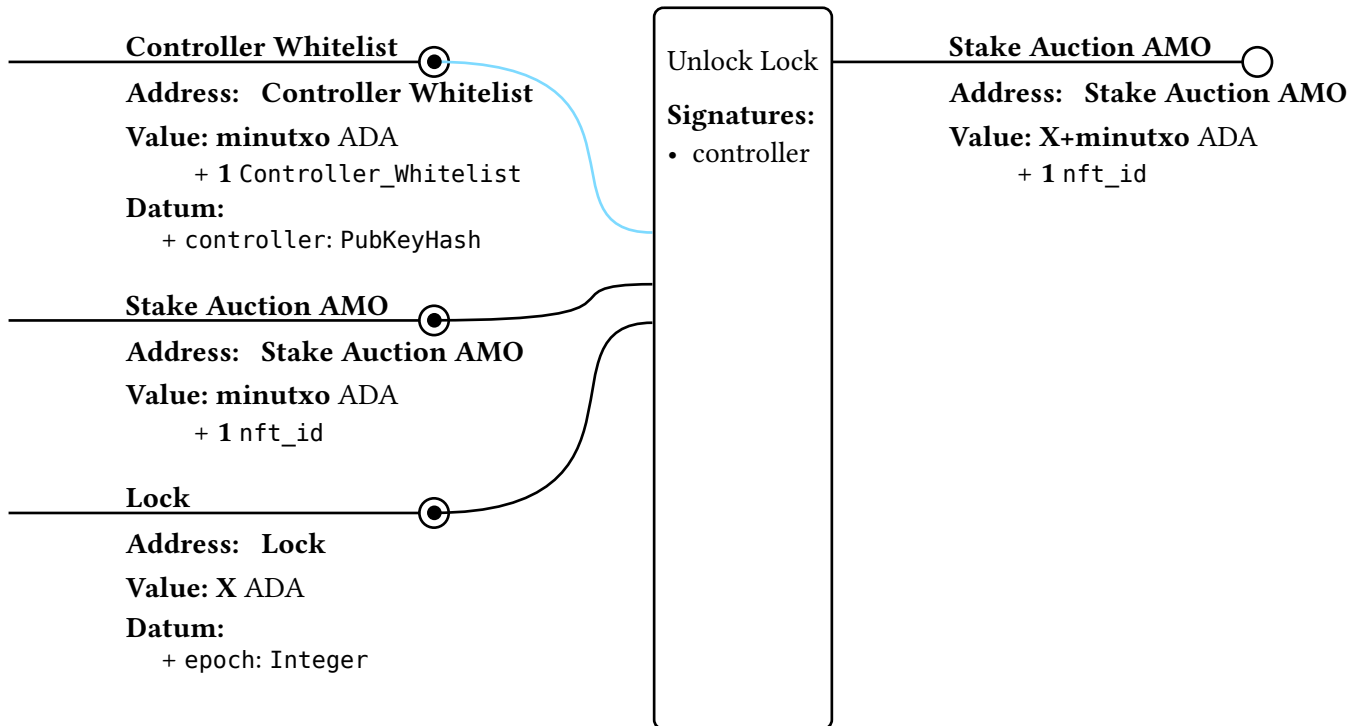
Can partial/fill multiple bids at once

$$X = B / (\text{apy}/73)$$

Lock has the current epoch in datum

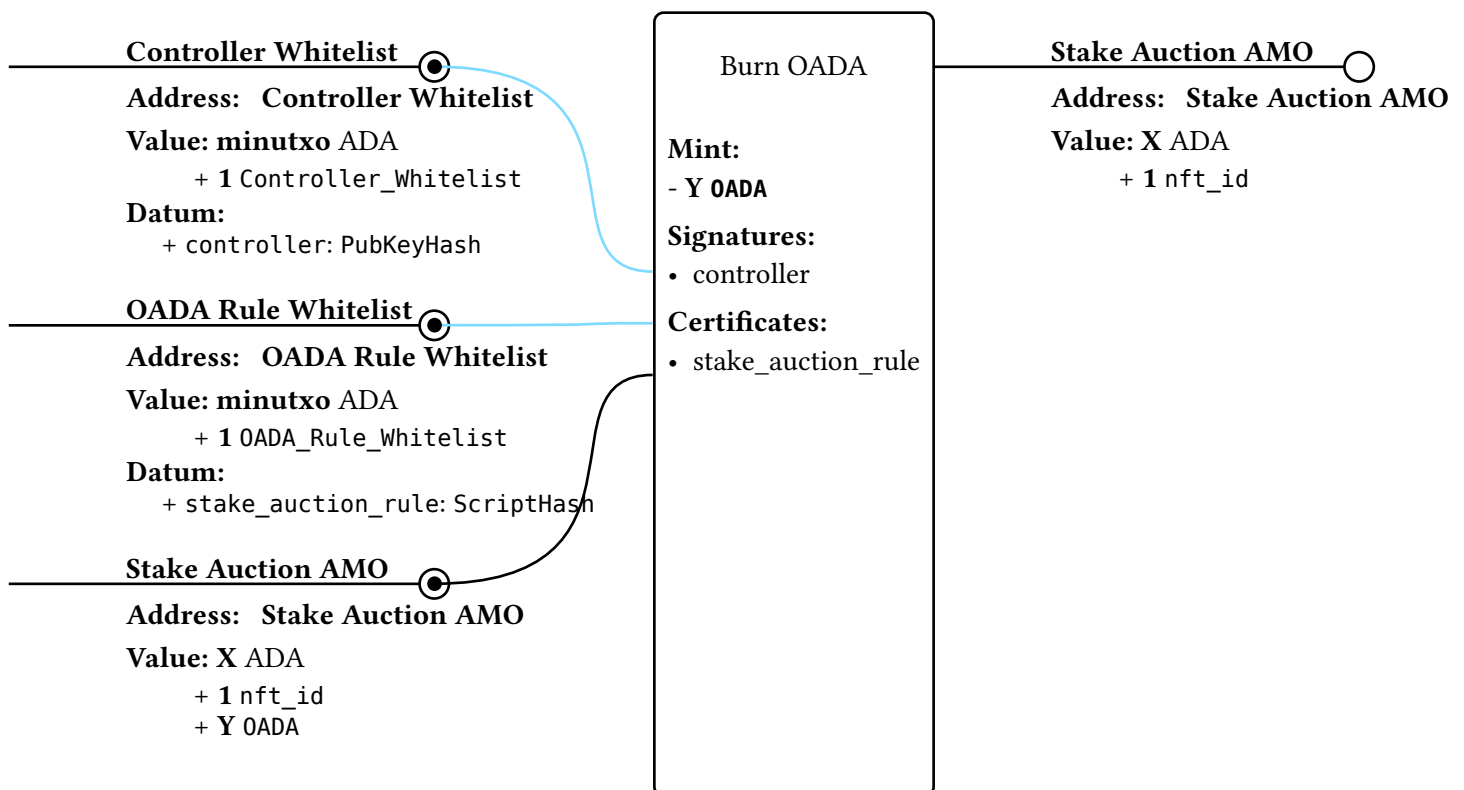
$\text{uncommitted_profit} += A$

7.30. Process expired Stake Locks



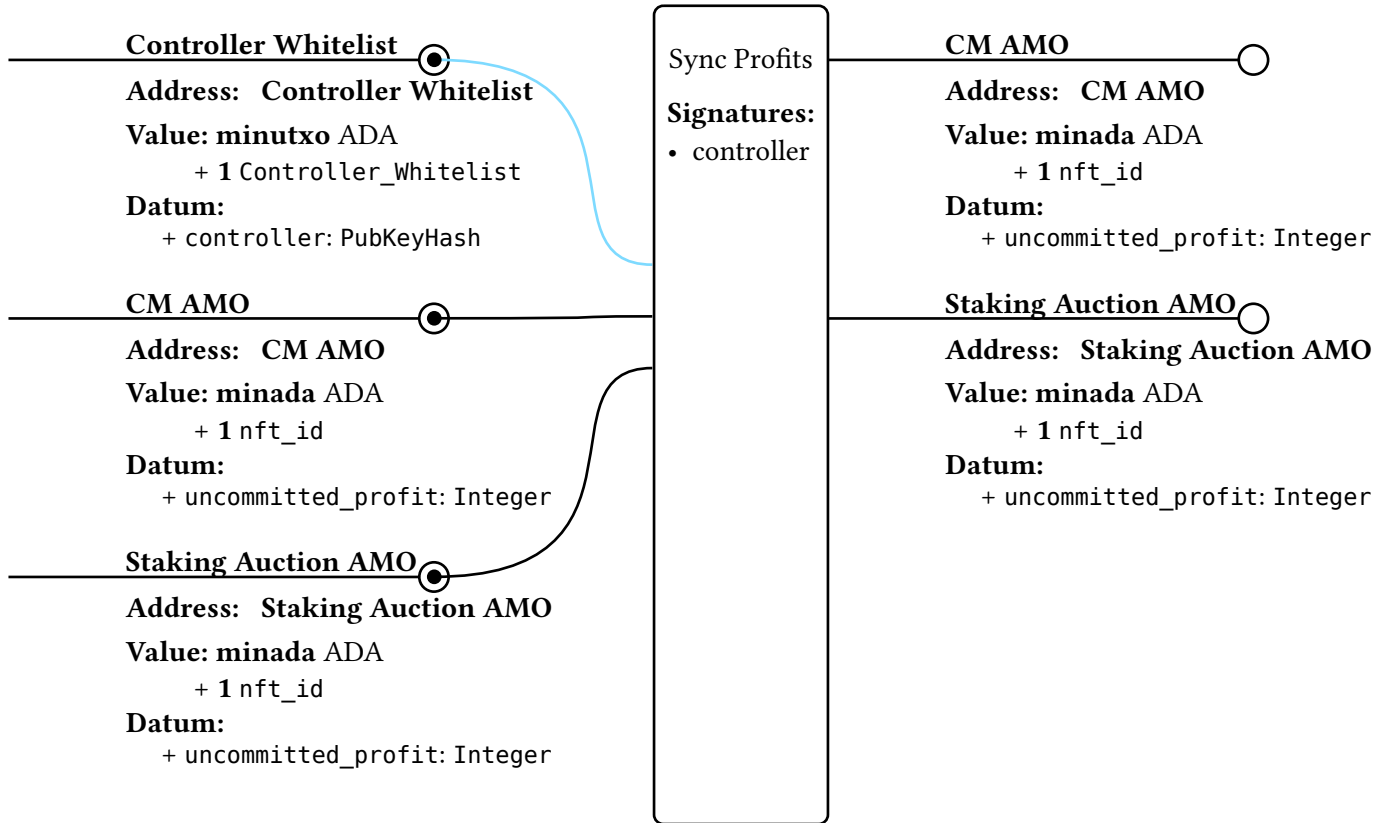
Note: Can unlock multiple at a time

7.31. Burn OADA from Bids

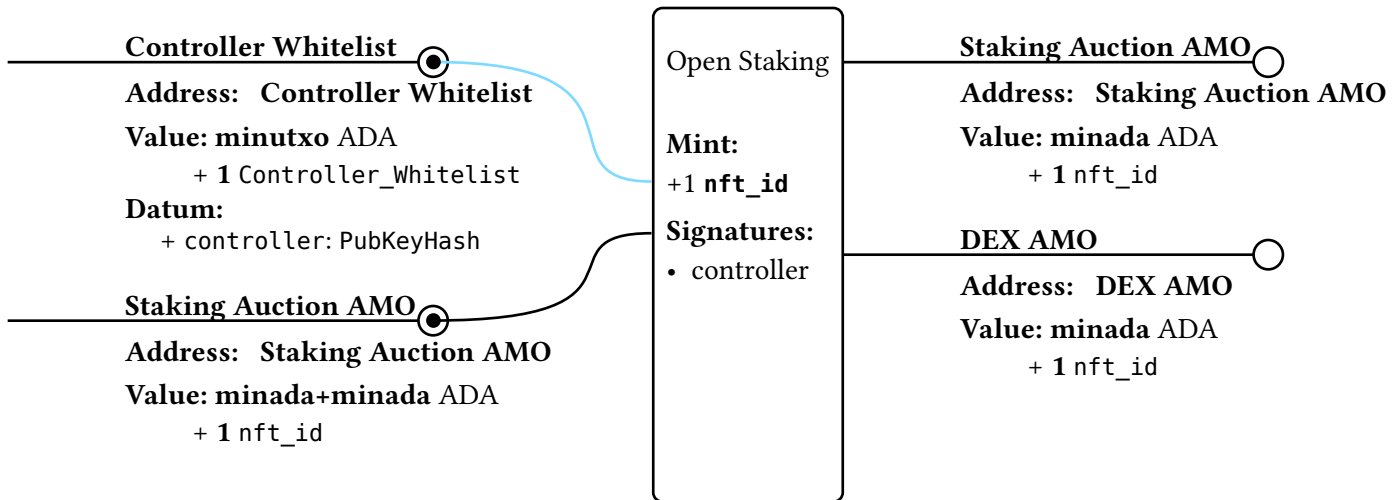


Note: Can unlock multiple at a time

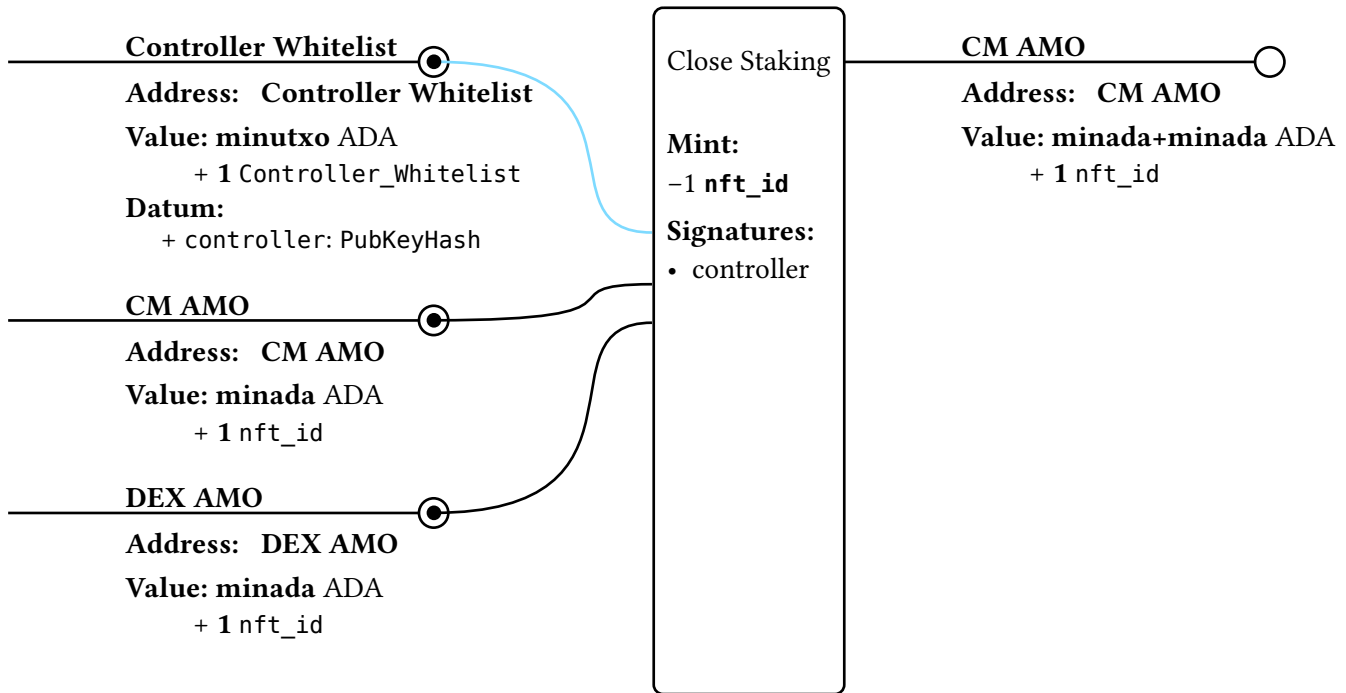
7.32. Sync Staking AMO Profits with the CM AMO



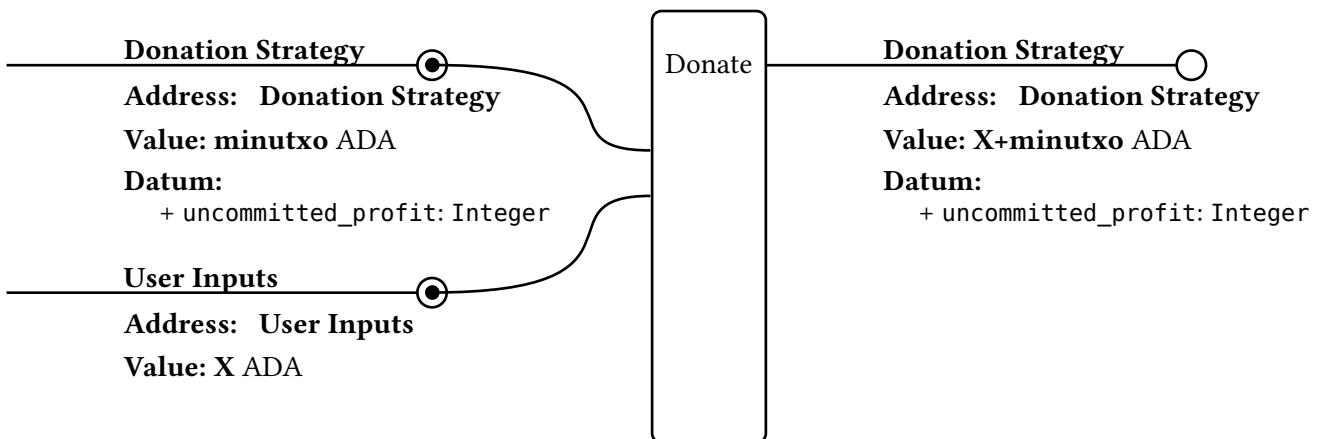
7.33. Open a Staking Auction AMO



7.34. Close a Staking Auction AMO

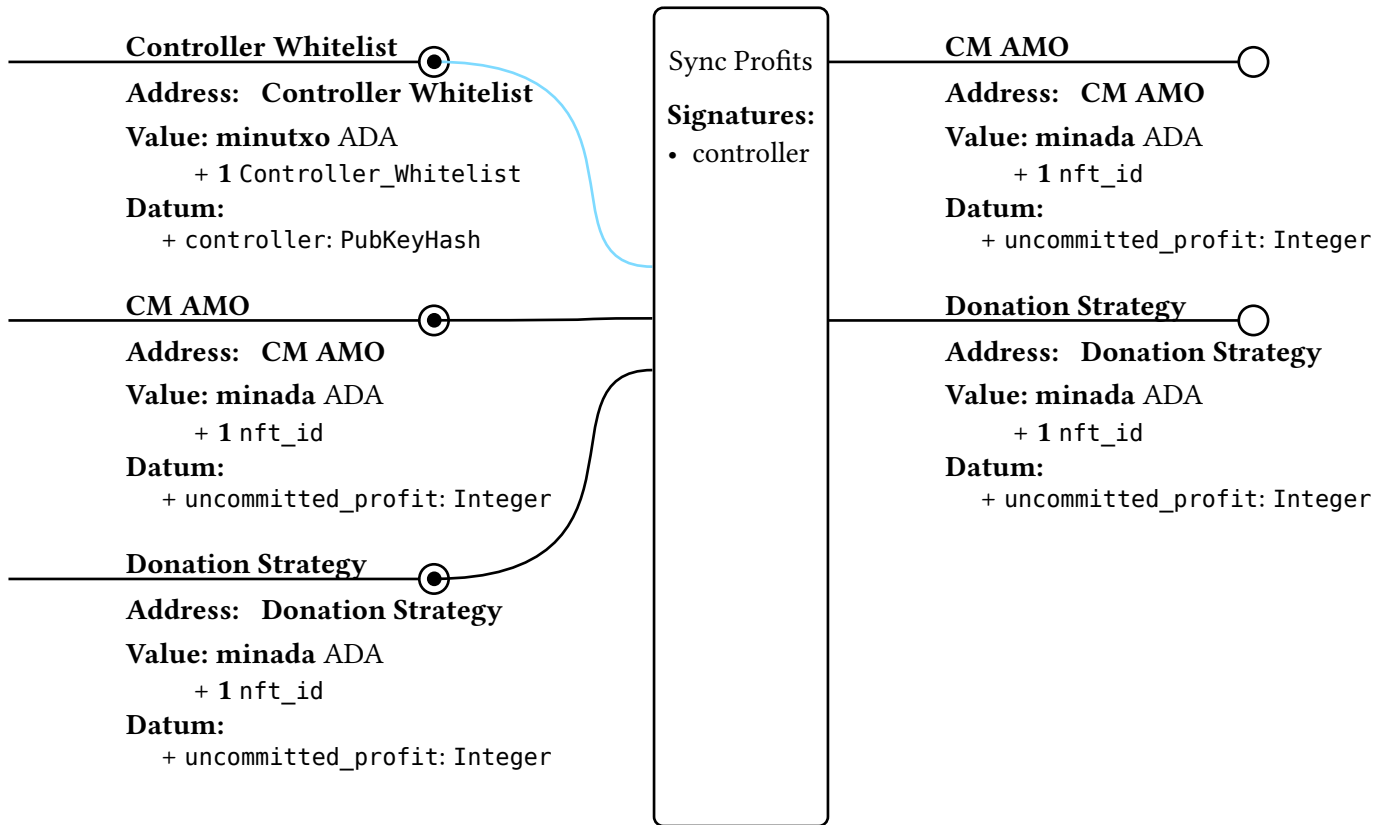


7.35. Donate via Donation Strategy

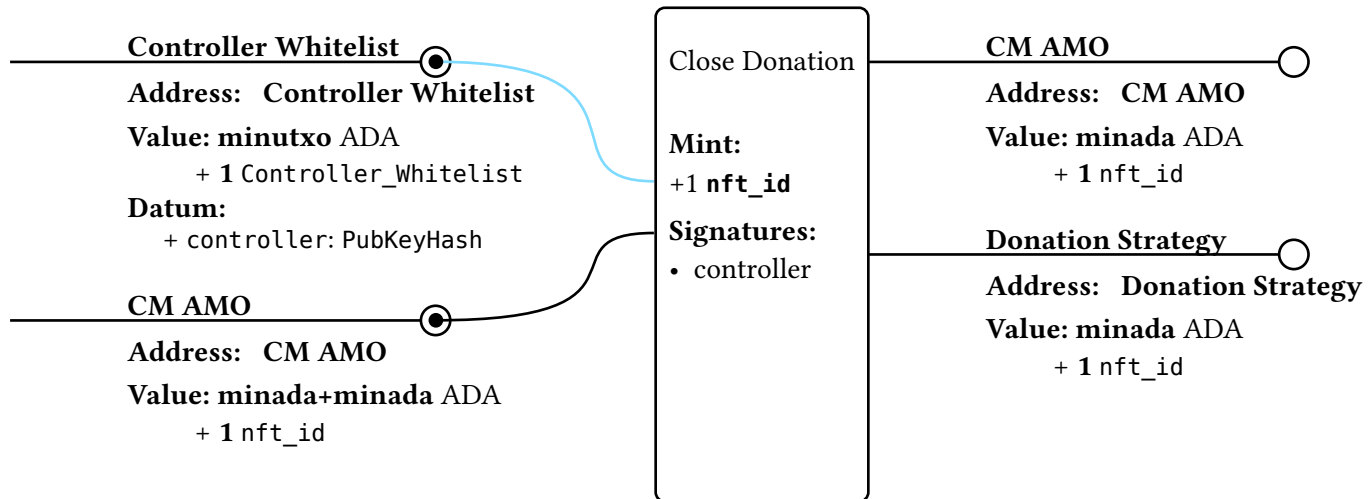


Note: $\text{uncommitted_profit} += X$

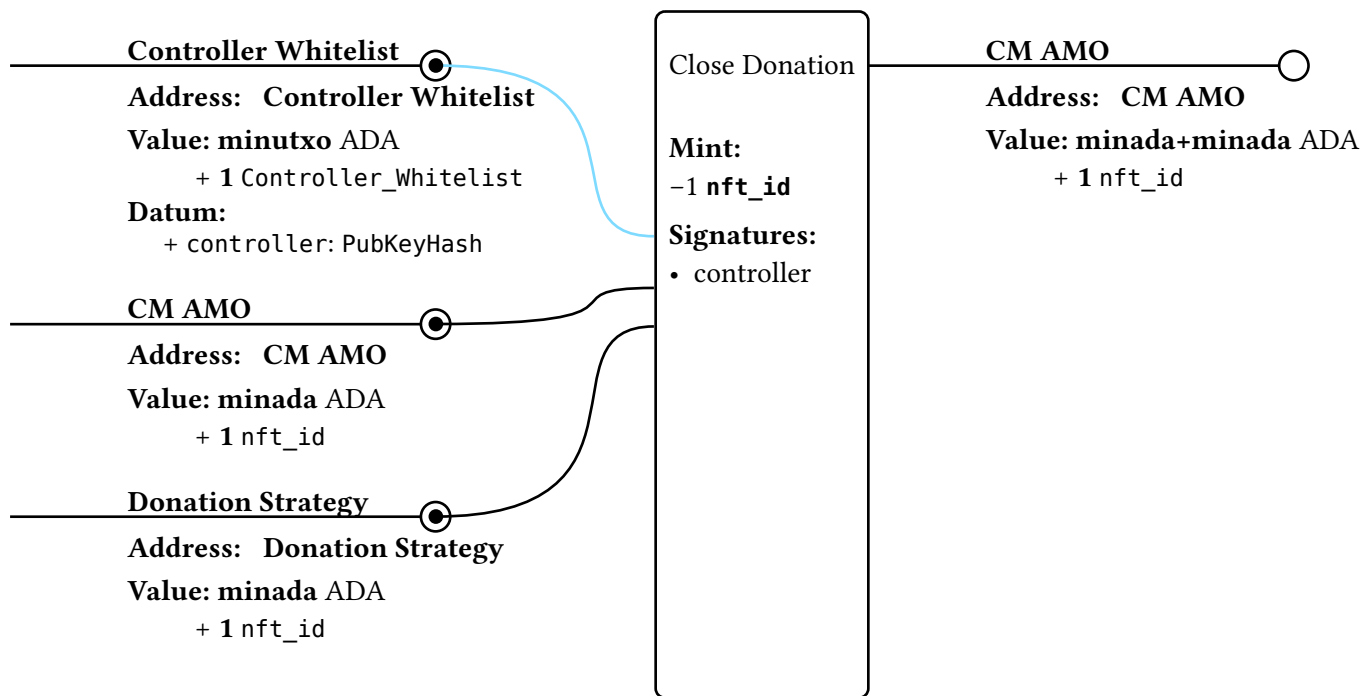
7.36. Sync Donation Profits



7.37. Open a Donation Strategy



7.38. Close a Donation Strategy



8. Deprecated Modules

No modules have been deprecated thus far

9. Changelog (System)

9.1. 1.0

- Added Forward Module
- Initial Version

10. Changelog (Document)

10.1. 2.0

- Documentation for V2

10.2. 1.0

- Initial Version